

南 开 大 学

本 科 生 毕 业 论 文（设 计）

中文题目： 南开大学 2021 年本科生毕业论文模板
——v1.2

外文题目： Graduation thesis template of Nankai University
——v1.2

学 号： 1710112

姓 名： 张鹏

年 级： 17 级

专 业： 统计学

系 别： 概率统计系

学 院： 数学与科学学院

指导教师： 魏雅薇 老师

完成日期： 2021 年 5 月

关于南开大学本科生毕业论文（设计）的声明

本人郑重声明：所呈交的学位论文，是本人在指导教师指导下，进行研究工作所取得的成果。除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人创作的、已公开发表或没有公开发表的作品内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。本学位论文原创性声明的法律责任由本人承担。

学位论文作者签名：

年 月 日

本人声明：该学位论文是本人指导学生完成的研究成果，已经审阅过论文的全部内容，并能够保证题目、关键词、摘要部分中英文内容的一致性和准确性。

学位论文指导教师签名：

年 月 日

摘 要

此模板根据《南开大学本科毕业论文（设计）指导手册 2018》（以下简称《指导手册》）的要求制作，这是目前最新的要求。由于作者专业的原因，本模板花较大篇幅展示了如何排版数学公式、定理和证明。希望此模板能帮助更多正在写论文的同学。

下载方式：

- 你可以在 GitHub: [NKU-biyelunwen-2021](#) 下载文件，在本地使用。若无法登录 GitHub，也可以在 [百度网盘](#) 下载，提取码：p4qv。
- 使用 Overleaf 的模板：[NKU-毕业论文-2021](#) 在线编程。

使用建议：

- 请使用 XeLaTeX 进行编译^①。
- 使用模板时，建议不要删除原来的内容，将其注释即可，以备日后需要。
- 若不希望将超链接上色，将 `preamble.tex` 中导入 `hyperref` 的 `colorlinks` 选项去掉（位于文件约 190 行）。
- 你需要自行在 `cover1.docx` 中制作封面并导出为 `cover1.pdf`，以替换作者的封面。不要使用作者的封面☹
- 推荐在 [Tables Generator](#) 网站制作表格，使用 [Mathpix](#)^② 软件书写数学公式，它们能很大地提升写作效率。

最后，由于本人水平有限，模板仍存在不足，欢迎指出，本人将会量力而为尽力完善。联系方式：(1) 微信：zp18102190105；(2) QQ：602795339；(3) 邮箱：602795339@qq.com。

关键词：南开大学毕业论文； $\text{\LaTeX}$ ；数学公式；定理环境；代码

^①在 Overleaf 上切换编译器的方式见 [文章](#)。

^②该软件经由阿里系某二手交易软件赋能后可发挥最大功效。

Abstract

If you skip the previous abstract in Chinese and come here for help, this template is not for you☺.

Keywords: L^AT_EX template; impatient author

目 录

一、 绪论	1
二、 相关工作	2
(一) 推荐系统研究现状	2
1. 传统推荐算法的原理与局限	2
2. 基于大语言模型的推荐研究进展	3
3. 现有研究的空白	3
(二) 多 Agent 系统研究	4
1. Agent 的基本概念与理论基础	4
2. 多 Agent 系统的分布式协作机制	4
3. 多 Agent 架构在推荐系统中的应用潜力	5
(三) 可视化辅助的 LLM 人机交互	5
1. 推理过程的可解释性可视化	6
2. 多维推荐结果的可视化比较	6
3. 对话交互中的视觉引导机制	7
三、 智能商品推荐系统	7
(一) 需求分析与方法设计	7
1. 需求层面分析	7
2. 系统总体框架设计	9
3. 三层品类分类体系设计	10
(二) 核心算法设计	11
1. 用户需求语义解析与提示工程约束	11
2. 异构商品多维评分模型	14
3. 双层场景化推荐引擎	17
4. 跨品类生态推荐算法	17
(三) 系统架构实现	20
1. 全栈基础架构与运行环境实现	20
2. 多 Agent 推荐模块实现	22
3. 品类识别与动态路由实现	27
4. 商品数据层实现与数据清洗	29
5. 用户界面与交互式可视化实现	32

(四) 本章小结	36
四、 系统评估	40
(一) 评估方案设计	40
(二) 核心功能逻辑验证	41
(三) 系统效能评估分析	41
(四) 从输入到图表的终局案例演示	42
1. 案例 1: 大学生的轻薄办公场景 (笔记本电脑)	42
2. 案例 2: 极致性能的游戏发烧场景 (跨品类生态组合)	43
3. 案例 3: 追求性价比的摄影入门场景 (相机)	44
(五) 本章小结	46
五、 本文贡献总结	46
(一) 提出面向消费场景的对话式推荐范式	46
(二) 构建兼顾灵活性与可靠性的双层推荐架构	46
(三) 实现跨品类协同推荐	46
(四) 以可视化手段增强推荐的可解释性	47
六、 局限性	47
(一) 商品数据的实时性不足	47
(二) 复杂语义场景下的解析局限	47
(三) 个体偏好的持续建模缺失	47
七、 后续改进方向	48
参考文献	49
致 谢	52

一、绪论

随着电子商务的快速发展，网络购物已深度融入人们的日常生活。据统计，中国网络零售市场规模已突破十万亿元，平台上在售商品数量动辄以亿计。商品数量的爆炸式增长虽然带来了更丰富的选择，却也让消费者在面对庞大的商品列表、繁杂的规格参数以及质量不一的用户评价时，常常难以迅速判断哪些商品真正符合自己的需求。研究显示，当信息量过大时，用户的认知负担会显著增加，决策效率随之下降，甚至可能出现决策回避或购买后的懊悔情绪。心理学上的“选择悖论”（Paradox of Choice）早已表明：选项越多，幸福感反而可能越低，决策质量也未必提升。换句话说，信息越多，决策反而越难。

这一问题在数码品类中尤为突出。以智能手机为例，消费者在选购时需要综合比较处理器架构、屏幕刷新率、相机传感器尺寸、电池容量与快充协议、系统生态等多个维度，而每个维度本身又有大量行业术语与品牌话术的包装，进一步提高了普通用户的理解门槛。如果用户同时关注多个价位段的商品，所需处理的信息量将以指数级增长，决策周期往往从数小时延长至数天，甚至因无所适从而放弃购买。

现有的推荐方式难以破解这一困境。**第一**，传统协同过滤依赖历史行为数据，对用户当下的即时意图理解不足，且在新用户、新商品场景下面临严峻的冷启动挑战；其推荐逻辑本质上是“你和相似的人买了什么”，而非“这款商品是否真的适合你当前的需求”。**第二**，基于大语言模型的纯对话式推荐虽然能理解自然语言意图，但由于缺乏与实时、权威商品库的深度绑定，容易产生“幻觉”——推荐出并不存在的型号、过时的参数或无法核实的性能描述，反而增加了用户的信任成本。**第三**，内容型测评平台（如知乎、什么值得买）信息丰富，创作者的深度测评具有较高参考价值，但此类内容依赖个体的主观经验，覆盖品类和更新频率均受限，用户仍需投入大量时间自行阅读、筛选与归纳，且不同测评之间的评价标准并不统一，横向比较困难。**第四**，平台自带的筛选与排序功能虽然支持按参数过滤，却无法理解诸如“拍照好用、续航够用、价格不超过三千”这类融合偏好与约束的自然语言表达，用户依然需要手动将模糊需求转化为精确筛选条件。

上述四类方式分别在“理解自然语言即时需求”“提供可靠结构化事实依据”和“以可视化方式辅助多维决策”三项关键能力上存在不同程度的缺失，且至今尚无一套成熟方案能够将这三项能力有机整合。这一空缺，正是本研究所要填补的核心技术挑战。

在上述背景下，本文提出并实现了一套基于多 Agent 架构的智能商品选购推荐系统。系统的核心设计思路在于：将大语言模型对自然语言的理解能力，与结构化

商品知识库中明确、可验证的商品数据深度结合，再通过三级分工的 Agent 路由体系，使用户一句自然语言的需求能够被准确拆解、精准匹配，并最终清晰直观的可视化方式呈现推荐结果。系统支持 10 大数码品类的精准推荐与未知品类的动态扩展，每次推荐给出三款候选商品并附带推荐理由与优缺点分析；同时通过多维雷达图与分组柱状图对候选商品进行可视化对比，将抽象的规格数据转化为直观的差异图景，有效降低用户的认知负担与决策门槛。

各章概述

第二章对推荐系统、多 Agent 技术以及可解释性人机交互的相关研究进行综述；第三章依据问卷调研结果开展需求分析，并给出包含数据采集流程在内的系统总体架构；第四章围绕双层推荐引擎、多维度评分机制和生态化推荐方法等核心算法展开说明；第五章介绍系统的具体工程实现；第六章通过功能测试与用户评价对系统进行验证；第七章对全文工作进行总结，并提出后续可能的优化方向。

二、相关工作

(一) 推荐系统研究现状

1. 传统推荐算法的原理与局限

现有的推荐方法大致可以分为三类，各有其形成背景与适用范围。

协同过滤（Collaborative Filtering, CF）是推荐系统中最早成型、也最具代表性的一类方法。无论是基于用户还是基于物品的实现，它们都建立在同一个朴素但有效的假设上：相似的用户会喜欢相似的物品 [2]。矩阵分解方法的引入显著提升了协同过滤在大规模数据上的表现，SVD++ 等模型在工业实践中得到广泛应用 [3]，让这一路线在工业界的实践相对成熟，尤其是在数据规模足够大、用户偏好稳定时，往往能给出相当可靠的结果。然而，这类方法本质上依赖历史交互，因此在数据稀疏的情况下往往力不从心；当新用户或新商品刚进入系统、缺乏足够行为记录时，“冷启动”问题会格外明显 [1]。此外，协同过滤并不善于理解用户当下的即时意图。例如，两位用户都可能浏览过同一类笔记本电脑，但其中一位正在寻找轻薄便携款，另一位关注的是游戏性能，这类需求差异不会自然地体现在历史行为里，也超出了协同过滤的辨别能力。

基于内容的过滤（Content-Based Filtering）试图绕开对其他用户行为的依赖，通过分析物品属性和用户过往偏好建立匹配关系 [4]，因此在用户规模较小时也能保持稳定性能。它的挑战在于对特征工程的高度依赖：人工构建的特征往往难以覆盖商

品的全部关键要素，且维护成本高；系统很容易陷入“越推越窄”的过滤气泡，难以拓展用户的潜在兴趣。此外，面对自然语言中隐含的偏好描述，例如“想要一个适合旅行用、充电久一点的相机”，传统内容过滤缺乏足够的语义理解能力，往往无法有效处理，因此常被质疑“听不懂人话”。

基于知识的推荐（Knowledge-Based Recommendation）则通过领域规则或知识图谱来约束匹配空间，在如数码产品、房产、汽车等高价值、低频决策场景中表现尤为稳健 [5][6]。它的优势在于决策链条清晰、可解释性强，可以明确指出某个推荐结果背后的逻辑依据。但这类方法的构建门槛较高：规则需要密集领域知识支撑，维护难度也随商品更新而增加；同时，系统面对开放式自然语言的表达缺乏弹性，扩展性相对不足。总体而言，它适合结构化、逻辑性明确的任务，却难以应对用户真实咨询场景中那种复杂、多变、带有模糊需求的对话式偏好表述。

2. 基于大语言模型的推荐研究进展

近年来，大语言模型（LLM）的出现为推荐系统带来了一条全新的技术路线。以 TALLRec、LLM4Rec 等模型为代表的一系列研究尝试将 LLM 置于推荐流程的“语义中枢”位置 [7][8]，让用户以自然语言表达的需求不再需要被手工拆解成标签或关键词，而是能够直接转化为更具语义结构的查询条件。例如，用户说“想买一台适合剪视频、预算八千左右的笔记本电脑”，模型可以自动从中提取预算、性能需求、用途场景等信息，进一步对接检索流程。与此同时，当缺乏用户历史行为时，LLM 还能通过语义推断生成合理的初步偏好解释，为缓解冷启动提供了新的技术可能 [7][8]，使得推荐不再完全依赖已有交互记录。

然而，将 LLM 直接引入推荐并非没有代价。最典型的问题是幻觉生成：模型可能会编造不存在的商品型号，误述产品规格 [8]，甚至在品类属性之间做出错误归纳。在数码消费等高度依赖真实参数和可验证信息的场景中，这些错误往往会导致明显的决策风险 [9]，用户难以容忍。此外，LLM 给出的推荐理由虽然往往听起来流畅自然，但并不总能清晰映射回可信的数据来源，用户无法判断其逻辑是否基于真实商品信息还是模型的“想当然”，导致实际应用上难以完全依赖。

3. 现有研究的空白

综合现有工作，可以看到几个尚未被充分解决的问题。其一，许多方法仍停留在“大模型接入推荐流程”的宏观层面，缺乏对具体使用情境的细致设计，难以让领域知识真正融入推荐链路，导致在专业类场景（如数码、家电、户外装备等）表现不

稳定。其二，跨品类的一致化路由机制仍不成熟：不同类别对商品属性的表达方式、用户关注点、约束条件差异显著，系统很难保持统一的理解与响应能力，导致推荐质量随品类波动。其三，缺少能够有效依托结构化数据的稳健机制，用以约束模型的生成边界、抑制幻觉，并确保每一次推荐都能基于真实、可验证的商品信息。

本文提出的架构正是围绕这些空白展开：通过在语义理解、品类路由和数据约束之间建立更紧密的联动机制，希望在真实使用场景中构建一个既能发挥大模型表达力，又具备可验证性与稳定性的推荐系统框架。

（二）多 Agent 系统研究

1. Agent 的基本概念与理论基础

在人工智能研究中，Agent 通常被描述为能够感知环境、自主作出判断并执行行动的软件实体 [10]。这一概念的核心在于 Agent 通过感知、推理与行动三个环节形成一个持续闭环：感知负责收集外部环境的状态变化；推理阶段整合内部模型与经验，对输入信息进行判断；执行环节将推理结果落实为具体动作，从而影响环境并进入下一轮循环。基于这一框架，研究者逐渐形成了不同的 Agent 设计思路。

反应式 Agent 强调对环境变化的快速响应，通常基于预设规则或简单的刺激-反应机制，在要求实时性高但情境较为明确的场景中表现稳定。相对而言，慎思式 Agent 更倾向于先构建内部模型，再通过规划、策略选择等步骤做出决策，适用于任务复杂、目标多元的情境。这两类 Agent 在灵活性、复杂度以及可解释性上呈现不同取向，也构成了后续多 Agent 系统设计的基本背景。

2. 多 Agent 系统的分布式协作机制

多 Agent 系统 (Multi-Agent System, MAS) 通过让多个具有自主能力的 Agent 协同工作，为处理复杂、动态或跨领域任务提供了一种可扩展的解决方案 [11]。在这一框架下，系统通常采用分布式任务拆分和角色划分的方式，让不同 Agent 在相对独立的职责范围内协作完成整体任务。这样不仅提升了系统在规模扩张时的可扩展性，也增强了面对局部故障或信息不确定性时的鲁棒性。

在众多协作模式中，层级式结构尤为常见。上层的协调 Agent 负责理解任务、判断意图并进行路由，而下层的领域 Agent 则依据专业知识处理更细粒度的问题。由于职责边界清晰，这一架构便于集成与扩展 [12]，与本文采用的 “MultiCategoryAgent + 品类专家 Agent” 模式天然吻合。

随着大语言模型的发展，面向多 Agent 协作的工程化框架也逐渐成熟。例如 LangChain、LangGraph 等系统通过路由链（Router Chain）、计划-执行结构以及工具调用接口，使开发者能够更方便地构建多 Agent 之间的信息流动和任务依赖。这些框架强调模块化与可组合性，让不同 Agent 能够在统一协议下协同处理复杂任务，为大型推荐系统的构建提供了实际可行的工程路径。

3. 多 Agent 架构在推荐系统中的应用潜力

近年来，随着对话式推荐、个性化推荐与可解释推荐的发展，多 Agent 架构开始在推荐系统领域展现出明显的优势。对话式场景中，用户的完整需求往往在多次对话中慢慢形成，多 Agent 系统中的对话 Agent、意图识别 Agent 和领域专家 Agent 能组建分工明确的协作链路，使用户的语义表达能够被不断细化并映射到更清晰的商品偏好结构。

在跨品类推荐中，传统单体模型往往难以同时处理数码、家电、户外装备等类别之间差异巨大的属性体系，而多 Agent 架构能够让每个品类由相应的专家 Agent 负责处理，使推荐逻辑更具一致性和可解释性。同时，工具调用能力让 Agent 可以在对话过程中实时访问结构化数据库、知识图谱或外部 API，从而有效弥补大模型在知识时效性与事实准确性上的不足。

一系列工作证明了多 Agent 在复杂情境下的潜力。Park 等人在 UIST 2023 提出的 Generative Agents 框架展示了具备记忆、反思与社交行为的类人 Agent 如何形成稳定协作 [14]；而 MetaGPT[13] 通过角色分配与任务流水线的方式，实现了更偏工程化的大规模协作能力。这些成果说明，在需要多轮推理、跨领域知识整合或实时信息检索的任务中，多 Agent 具备良好的适应性。这些特性为构建更稳健、可解释、可扩展的推荐系统提供了重要的参考方向，也为本文提出的系统架构奠定了理论与工程基础。

（三）可视化辅助的 LLM 人机交互

随着大语言模型逐渐被应用于推荐与决策支持场景，单纯依赖文本生成的交互方式逐渐暴露出一定局限。一方面，模型的推理过程对用户而言难以观察，推荐结果缺乏透明度；另一方面，当系统返回多个候选方案时，用户需要在不同属性之间进行比较，也容易产生较高的认知负担。同时，对于需求尚不明确的用户，传统对话界面往往缺乏有效的引导机制。近年来的研究开始尝试将信息可视化方法引入 LLM 交互界面，通过图形化方式辅助理解模型推理过程、比较推荐结果，并在交互过程

中提供适当引导。围绕这一思路，相关工作大体可以从三个方面展开：推理过程的解释性呈现、推荐结果的可视化比较以及交互过程中的视觉引导机制。

1. 推理过程的可解释性可视化

对于普通用户而言，大语言模型的推理过程通常难以直接观察，推荐结论往往以最终文本形式呈现。这种“黑盒式”输出在一定程度上影响了用户对系统结果的信任。可解释性人工智能（Explainable AI, XAI）研究正是试图缓解这一问题，其中一种常见思路是通过可视化方式呈现模型决策的依据或推理路径 [15]，使用户能够理解推荐结论形成的基本逻辑。

Wang 等人在对话推荐系统研究中发现，如果系统能够以结构化步骤展示推荐理由，例如“因用户提及‘轻巧’，过滤掉重量超过 1.5kg 的机型”，用户对系统的信任度指标（Trust Score）明显高于仅给出文本推荐的情况 [16]。Strobel 等人在 CHI 会议提出的注意力权重热力图可视化工具，则通过标注模型在输入文本中的关注区域，使非专业用户也能够直观地看到模型关注的关键词位置 [18]，这为 LLM 推荐系统的解释性设计提供了新的思路。另一类研究尝试将思维链（Chain-of-Thought, CoT）推理过程进行图形化表达 [19]，例如将逐步推理转换为流程节点或关系结构，从而使模型的推理过程能够以更直观的方式呈现。这类可视化形式不仅有助于系统开发者调试模型，也为用户理解与监督模型行为提供了可能。

2. 多维推荐结果的可视化比较

在实际推荐场景中，系统往往会返回多款候选产品。用户需要在价格、性能、重量、续航等多个维度之间进行比较，如果仅通过文本描述呈现信息，往往难以快速形成整体判断。信息可视化研究表明，合适的图表形式能够降低用户在多属性信息处理过程中的认知负荷（Cognitive Load），并有助于提升决策效率 [20]。

Amar 等人 [20] 在信息可视化研究中总结了用户分析多维数据时常见的认知任务，包括数值检索、条件筛选、趋势识别以及异常值判断等。这些任务为面向决策的图表设计提供了重要参考。在多属性比较场景中，** 极坐标雷达图（Radar Chart）** 常用于展示不同对象在多个指标上的整体表现，它能够在同一视图中呈现各产品在不同维度上的相对优势与劣势，从而帮助用户快速获得整体印象 [21]。相比之下，当分析重点集中在某一具体维度时，** 分组柱状图（Grouped Bar Chart）** 通常能够提供更清晰的数值对比效果 [20]。在对话式推荐系统中，这类图表如果与 LLM 生成的推荐理由结合使用，可以同时提供数据层面的客观比较与语言层面的解释说明，

从而形成更加完整的决策支持信息结构。

3. 对话交互中的视觉引导机制

在许多实际使用情境中，用户在初次提出问题时往往并未形成明确需求。如果界面仅提供自由文本输入，用户可能难以准确表达自己的意图，进而导致多轮低效对话。为改善这一问题，人机交互（HCI）领域开始尝试在对话界面中加入可视化交互元素，通过轻量级的视觉提示帮助用户逐步明确需求。

Amershi 等人 [22] 在 IUI 会议的研究指出，在对话界面中加入结构化视觉提示，例如约束条件标签或意图确认卡片，可以在交互过程中帮助用户逐步细化需求，从而减少不必要的对话轮次。Cai 等人 [23] 提出的生成式 UI（Generative UI）概念则进一步扩展了这一思路，即根据当前对话上下文动态生成不同的界面组件，例如筛选条件、参数滑块或结果卡片。这种动态界面形式在探索型信息检索任务中表现出较高的用户满意度和任务完成率。与此同时，界面视觉设计本身也会影响用户对系统的初始感知。例如“玻璃拟态（Glassmorphism）”等现代界面风格以及平滑的过渡动效，已被相关可用性研究证明能够在一定程度上降低用户接触 AI 系统时的心理门槛，并提升感知可用性（Perceived Usability）与整体交互体验 [24]。

三、智能商品推荐系统

（一）需求分析与方法设计

1. 需求层面分析

在设计智能商品推荐系统之前，有必要首先了解用户在电商选购过程中所面临的实际问题。为此，本文围绕数码电子产品选购体验开展了初步问卷调研，共回收有效问卷 20 份，调研内容主要涵盖信息获取、产品比较以及决策判断等环节的使用体验。

调研结果显示，用户在购买数码产品时普遍需要同时参考多个信息渠道。约 85% 的受访者表示，在查询商品信息时通常需要在京东、B 站、小红书等平台之间来回切换，由于各平台的信息呈现方式不统一，用户往往不得不手动整理参数并自行完成横向对比，这一过程显著拉长了决策周期。与此同时，约 80% 的受访者认为网络评测内容普遍带有一定主观倾向，部分文章或视频夹杂个人偏好甚至商业推广因素，使得用户难以从中提取真正客观的产品信息。此外，部分技术参数本身具有一定的理解门槛——“传感器单像素尺寸”“阻抗”等专业术语对普通用户而言较难直观把握，

约 40% 的受访者明确表示希望系统能够通过图表等可视化方式辅助理解关键参数的差异。

为系统性梳理上述调研发现，表1从功能需求与非功能需求两个维度归纳了系统的设计目标。

表 1 智能商品推荐系统需求汇总

需求类型	需求项	具体描述
功能需求	自然语言理解	解析含预算、场景、品牌偏好等信息的口语化查询
	多品类推荐	支持手机、笔记本、相机等 10 类数码产品的精准推荐
	结构化筛选	依据价格上限、品牌偏好等硬性约束过滤候选商品
	推荐理由生成	用自然语言解释每款推荐产品的核心优势与适用场景
	可视化对比	以雷达图、柱状图呈现多维度性能差异，辅助用户决策
	跨品类搭配	识别用户已有设备，给出生态化配套建议
非功能需求	响应时延	端到端推荐流程耗时控制在 5 秒以内
	识别准确率	预设品类关键词识别准确率达 100%
	可扩展性	支持动态添加新品类而无需修改核心推荐链路
	可解释性	每条推荐均有结构化数据来源可追溯，抑制幻觉生成

上述问题共同指向三类功能需求。在输入层面，系统需要能够理解用户较为口语化的需求表达，例如"3000 元拍视频的相机" 这类描述实际上同时包含预算范围、使用场景与产品类型等信息，系统须借助大语言模型完成语义解析与槽位填充，从自然语言中提取结构化参数。在处理层面，调研显示大多数用户希望系统在一次交互中给出 3 至 5 款重点产品并附带简要推荐理由，因此系统需要基于客观参数完成数据筛选，并生成具有解释性的推荐说明，而非简单罗列商品列表。在输出层面，系统应提供可视化展示以帮助用户理解产品间的性能差异；考虑到数码产品日益形成

生态化使用趋势，系统还应能够提供跨品类设备的搭配建议。综合而言，系统设计需要兼顾通用性与客观性，并以关系型数据库作为底层事实来源，以减少大模型在推荐过程中可能产生的幻觉问题。

2. 系统总体框架设计

在明确上述需求之后，本文进一步构建了支撑这些功能的整体架构。系统以用户输入自然语言请求为起点，通过多 Agent 协同机制完成全流程任务处理。

用户输入首先由前置模块进行语义识别，通过关键词检测或调用大语言模型完成场景分类；识别出具体商品类别后，中央调度模块结合当前会话上下文将任务路由至对应的领域专家 Agent。领域 Agent 随后解析用户需求，提取预算范围、使用场景等关键信息，并通过 ORM 访问底层关系型数据库，先按硬性参数筛选，再结合场景规则匹配，生成候选商品列表。在确定候选产品之后，系统调用大语言模型生成推荐理由，同时将评分数据传递至可视化模块，用于生成雷达图或分组柱状图，最终以图文结合的形式呈现于前端界面。

在软件实现层面，系统整体架构划分为四个相对独立的模块层级。最外层为表象展示与会话层，负责前端界面展示和用户会话管理，动态加载图表组件与商品信息卡片；第二层为路由与意图解析层，将自然语言输入转换为结构化参数，通过对模型输出格式加以约束，保证业务层能够直接读取相关数据；第三层为业务决策与领域代理层，执行场景匹配、评分计算与候选产品排序等核心推荐逻辑，并生成最终的推荐说明；最底层为实体映射与事实数据层，存储结构化商品参数数据，目前覆盖多个数码产品类别并包含数千条参数记录，严格的数据结构设计有效约束了模型产生幻觉的空间。

此外，架构中还引入了动态领域扩展机制以提升系统的可扩展能力。当系统检测到用户需求属于尚未预置的新商品类别时，可调用大语言模型对该类别的评价维度进行初步推断，临时构建相应的评分体系，并自动切换至咨询模式以保证用户仍能获得基础信息服务。这一机制使系统能够在不修改既有代码的情况下扩展所支持的商品类别范围。

从工程实现视角看，四层架构与项目代码模块存在明确的对应关系，如表2所示：

系统的完整数据流可归纳为以下六个阶段：① 用户在前端输入自然语言需求；② CategoryDetector 识别商品品类，MultiCategoryAgent 路由至对应领域 Agent；③ 领域 Agent 通过提示词工程完成结构化意图解析，提取预算、场景、品牌偏好等槽位信息；④ 双层推荐引擎执行候选召回（场景预设优先 + 全库兜底备用）与多维度

表 2 系统层次架构与代码模块对应关系

架构层	核心文件	主要职责
表象展示与会话层	app.py	前端界面渲染、会话状态管理、图表与商品卡片动态加载
路由与意图解析层	multi_agent.py、category_detector.py	品类识别、Agent 路由分发、动态代理实例化
业务决策与领域代理层	base_agent.py、electronics_agents.py	意图解析、候选召回、评分排序、推荐理由生成
实体映射与事实数据层	models.py、SQLite 数据库	ORM 模型定义、参数存储、结构化数据管理
公共组件	visualizer.py、config.py、ecosystem_config.py	图表绘制、全局配置、生态场景定义

排序；⑤ LLM 生成推荐理由，可视化模块构建多类型交互图表；⑥ 前端组装商品信息卡片与图表，以图文结合形式反馈给用户。跨品类请求还会在步骤 ③ 之后额外触发生态增强模块，识别用户的潜在配套需求。

3. 三层品类分类体系设计

品类的准确识别是推荐流程的前提条件。系统在 category_detector.py 中构建了一套三层品类分类体系，将 10 个数码产品品类按功能属性组织为大类-中类-小类的层级结构，并为每个品类附加关键词词典与生态标签。

- **中类一：核心电子设备**——手机、笔记本电脑、平板电脑；
- **中类二：影音设备**——相机、耳机、蓝牙音箱；
- **中类三：数码配件**——智能手表、显示器；
- **中类四：游戏设备**——游戏主机、显卡。

每个品类均配有多组关键词列表，兼顾通俗称谓与英文缩写，用于快速模糊匹配。例如笔记本品类覆盖"笔记本""电脑""laptop""notebook""macbook""游戏本"等

词汇；耳机品类覆盖"耳机""headphone""airpods""耳麦"等。当用户输入无法被预设关键词命中时，系统退回至大语言模型进行语义级别的品类推断，保证对长尾表达的覆盖能力。

此外，每个品类还携带若干"生态标签" (ecosystem_tags)，用于跨品类推荐阶段的场景索引。例如笔记本电脑关联"移动办公""专业游戏""摄影创作"三个场景，相机关联"摄影创作"场景，显示器同时关联"移动办公""专业游戏""摄影创作"三个场景。这些标签构成了生态推荐模块的索引骨架，使系统在识别到当前品类后能够快速定位潜在的配套设备候选范围，并在完成主品类推荐后生成有针对性的跨品类搭配建议。

(二) 核心算法设计

1. 用户需求语义解析与提示工程约束

在智能商品选购系统中，用户需求的准确理解是推荐流程的起点。现实场景下，消费者通常通过自然语言表达需求，而这些表达往往具有明显的口语化特征，例如“我想买一台不差钱、能打游戏还能做后期修图的电脑”。此类描述同时包含主观评价和多个使用场景，因此难以直接转换为数据库查询条件。

虽然大语言模型在语义理解方面具有显著优势，但在长文本推理过程中仍可能出现注意力漂移或语义扩散问题。如果在解析需求时缺乏理论与系统层面的约束机制，模型极易产生不稳定甚至虚构的参数，从而影响推荐系统的可靠性。Brown 等人在 GPT-3 的研究中验证了通过上下文示例引导模型完成下游任务的泛化有效性 [25]，Liu 等人则在提示工程综述中进一步指出，提示语承担了与任务约束格式和应用领域知识直接交互的核心作用 [26]。

循此思路，本系统将提示词的稳定构建视为工程层面的核心设计，采用“角色设定与结构化输出限制”的复合策略。在系统单轮交互中，模型首先被赋予“你是一个 {category_name} 导购专家”的角色指令。White 等人在软件工程视角的提示模式 (Prompt Pattern) 研究中分析表明，设定专家角色 (Persona) 能有效收缩模型的生成空间，使输出贴合该垂直领域的专业语境 [27]。继而，基于运行时的动态提示词拼接，系统要求模型完成 8 个维度的关键参数提取：1. usage (用途)；2. budget_level (预算描述)；3. max_price (数值预算，检测到“不差钱”等表达时映射为最高限额 999999，缺省默认为 20000)；4. sort_field (依据用户偏好推断的排序权重)；5. summary (核心诉求归纳)；6. product_type (特定类型约束)；7. brand (品牌偏好)；8. owned_items (用户已购设备，用于生态联动)。提取出的 owned_items 字段是实

现跨品类关联逻辑的核心，使系统能够识别用户已有的产品并推荐具有协同效应的新型号。

针对原生大模型输出格式不可直接作为程序输入的风险，系统于提示词末尾应用了“输出自动化器”设计理念 [27]。通过“输出严格 JSON”的数据契约声明，系统成功将自然语言处理环节转换为精确、可用于底层校验模块的数据传递。

```
1 def _parse_intent_generic(self, user_msg: str, category_name: str
    , fields_desc: str) -> dict:
2     system_rules = f"""
3     你是一个{category_name}导购专家，负责从对话中提取结构化信息。
4     【待提取维度】：
5     1. usage（用途）：使用场景关键词
6     2. budget_level（投入）：预算描述
7     3. max_price（预算）：数字，默认 20000，"不差钱"设为 999999
8     4. sort_field（排序字段）：匹配最合适的评分字段（{fields_desc
    })
9     5. summary（摘要）：核心诉求归纳
10    6. product_type（类型）：显式要求的特定产品类型
11    7. brand（品牌）：用户明确想要购买的品牌（排除已拥有设备品牌）
12    8. owned_items（已购设备）：用户提到的已拥有产品型号/品类
13
14    输出严格 JSON：{"max_price": int, "sort_field": str, "
    summary": str, ...}}
15    """
16    # 调用 LLM 接口进行 JSON 提取
17    return self.call_llm_json(system_rules, user_msg)
```

代码 1 通用意图解析系统提示词构建（对应文件：base_agent.py）

在工程实现层面，意图解析还需应对多轮对话中的语境延续问题。用户在追问或修改需求时（如“再便宜一点的型号”或“改成黑色的”），往往省略了前几轮已确立的品类和核心约束。系统通过将最近两轮对话历史（history[-2:]）拼接至提示词消息队列，使 LLM 在解析当前查询时能够回溯先前上下文，从而正确继承前序轮次的槽位信息，避免重复询问。同时，为应对 LLM 偶发的输出格式偏差（如返回 Markdown 代码块包裹的 JSON），系统设计了容错预处理逻辑，在调用 json.loads() 之前自动剥离反引号与语言标识符。当解析彻底失败时，系统回退至安全默认值——预算上限 20,000 元、排序字段 Value_Score、摘要“综合推荐”——以保证推荐流程不会

因单次解析异常而中断。

此外，为解决系统预设商品类别有限的问题，系统设计了动态品类补全机制。当核心识别模块 `CategoryDetector` 检测到用户需求涉及未预置领域的商品时，系统将进入大语言模型知识推理模式，自动补充构建该品类的评价维度框架。

在动态推理流程的实现中，提示模板应用了“思维链”诱导策略。Wei 等人在研究中发现，要求模型隐式或显式地输出逻辑推理的中间步骤，可以成倍提升其应对复杂业务推理的容错率与准确性 [19]。基于此，构建算法（参见代码 2）并非单纯向大模型索要最终结果，而是引导模型逐步推演：首先提取该品类 3-5 个核心功能的评价指标（例如厨房电器的“功率”“容量”），进而为各个指标分配权重、生成底层数据模型的字段名，最后归纳出适合匹配用户口语表达的使用场景。依托这种步步推进的思维链模式扩展边界，系统即便并未预先创建底层库表，亦可保持推荐架构评价体系在处理长尾类别时的鲁棒性与语义自治。

```
1     def build_scoring_system(  
2         self,  
3         category_key: str,  
4         category_name: str,  
5         user_context: str = ""  
6     ) -> Dict:  
7         """  
8         LLM 动态构建该品类的评分体系  
9  
10        Args:  
11            category_key: 品类英文标识  
12            category_name: 品类中文名称  
13            user_context: 用户上下文（可选）  
14  
15        Returns:  
16            包含评分维度和场景预设的字典  
17        """  
18        system_prompt = f"""  
19        你是一个{category_name}领域专家。请为该品类设计一套评分体  
20        系和使用场景。  
21  
22        要求：  
        1. 设计 3-5 个核心评分维度（如性能、续航、便携性等）
```

```

23         2. 每个维度需要：中文名称、英文字段名(xxx_Score 格式)、权
           重(0-1, 总和=1)、描述
24         3. 设计 3-5 个典型使用场景及对应的推荐关键词
25
26     输出 JSON 格式：
27     {{
28         "dimensions": [
29             {{ "name": "性能", "field": "Performance_Score", "
           weight": 0.3, "description": "..."}}
30         ],
31         "scenarios": {{
32             "gaming": {{ "keywords": ["游戏", "电竞"], "
           presets": ["推荐品牌/型号关键词"]}}
33         }},
34         "default_sort_field": "主排序字段名"
35     }}
36     ""

```

代码 2 基于 LLM 的动态评分体系思维链构建算法 (对应文件: category_detector.py)

2. 异构商品多维评分模型

在完成用户需求解析后，系统需要对数据库中的商品进行统一的性能评价。然而，不同类别电子产品的参数体系差异较大，例如手机关注摄像头和电池容量，而耳机更强调频响范围和降噪能力。因此，不同类别之间难以直接进行性能比较。

为解决该问题，系统构建了双层评价维度体系。第一层为通用评价维度(Universal Dimension)，用于描述所有商品共享的基础指标，涵盖了综合性价比(Value_Score)、品牌信誉(Brand_Score)以及用户真实评价评分(User_Rating)。这类通用维度为不同类别的商品提供了统一的价值比较基准。

第二层为专有性能维度(Specific Dimension)，用于描述垂直领域产品的核心技术性能指标。例如在相机代理模块 CameraAgent 中，系统定义了“便携性评分(Portability_Score)”“低光画质(LowLight_Score)”和“视频能力(Video_Score)”；而在显卡代理 GPUAgent 中，则聚焦于“游戏性能(Gaming_Score)”“创作性能(Creative_Score)”及“功耗散热(Thermal_Score)”等指标。

算法 1 展示了 ScoringDimension 数据类的结构设计。该类以 name (中文展示标签)、field (数据库字段名)、weight (归一化权重) 和 description (维度说明)

算法 1 异构商品专有评分维度定义抽象（对应文件：base_agent.py）

```
1: Data Class ScoringDimension:  
2:   name: String {例如: “便携性”}  
3:   field: String {例如: “Portability_Score”}  
4:   weight: Float {权重区间 [0, 1]}  
5:   description: String {维度功能描述} =0
```

四个属性描述一个评分维度，构成系统中所有加权评分的基本计算单元。通过将权重与字段名分离存储，算法支持在不修改查询逻辑的前提下灵活调整各维度的重要程度。

算法 2 CameraAgent 专有评分维度示例

```
1: Class CameraAgent Inherits BaseDbAgent:  
2: Function GetSpecificDimensions() → List<ScoringDimension>:  
3:   Return [  
4:     ScoringDimension(“便携性”, “Portability_Score”, 0.3, “...”),  
5:     ScoringDimension(“低光画质”, “LowLight_Score”, 0.4, “...”),  
6:     ScoringDimension(“视频能力”, “Video_Score”, 0.3, “...”)  
7:   ] =0
```

算法 2 展示了 CameraAgent 的品类特定维度定义。相机品类选取便携性（权重 0.3）、低光画质（权重 0.4）和视频能力（权重 0.3）三个维度，其中低光画质权重最高，反映了当前无反相机用户对高感光性能的核心诉求。三个维度权重之和恰好为 1.0，确保加权得分的可比性，这一配置逻辑同样适用于系统中其他 9 个品类代理的维度定义。

由于商品原始数据来源多样，参数量纲与格式存在显著差异，系统在数据处理阶段引入了多类型归一化策略。对于布尔型参数，系统采用固定权值映射；对于离散枚举变量，根据性能等级设置分档权重；对于连续数值变量，则采用 Min-Max 归一化方法进行标准化处理。

归一化后的各项指标依据预设权重通过加权求和（Weighted Sum）计算综合得分，并映射至 0–100 的评分空间。该结果不仅用于最终推荐排序，也为前端 Plotly 可视化模块提供了稳定的数据结构，支持雷达图与多维能力对比图的动态呈现。

在数学形式上，商品 p 的综合推荐得分 $S(p)$ 可表示为各评分维度的加权求和：

算法 3 GPUAgent 专有评分维度示例

```
1: Class GPUAgent Inherits BaseDbAgent:  
2: Function GetCategoryConfig() → CategoryConfig:  
3:   Return CategoryConfig(  
4:     ScoringDimensions = [  
5:       ScoringDimension(“游戏性能”, “Gaming_Score”, 0.35, “...”),  
6:       ScoringDimension(“创作性能”, “Creative_Score”, 0.25, “...”),  
7:       ScoringDimension(“功耗散热”, “Thermal_Score”, 0.20, “...”),  
8:       ScoringDimension(“性价比”, “Value_Score”, 0.20, “...”)  
9:     ]  
10:   )=0
```

$$S(p) = \sum_{i=1}^N w_i \cdot \hat{s}_i(p), \quad \sum_{i=1}^N w_i = 1 \quad (1)$$

其中 N 为当前品类的评分维度总数（通用维度 3 个 + 专有维度 2 至 4 个）， w_i 为第 i 个维度的预设权重， $\hat{s}_i(p) \in [0, 100]$ 为归一化后的维度得分。

针对不同数据类型的参数，系统采用三种归一化策略：

1. **连续数值型参数**（如重量、电池容量、传感器尺寸）采用 **Min-Max** 归一化：

$$\hat{s}(x) = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \times 100 \quad (2)$$

对于越小越好的参数（如机身重量、功耗 TDP），在归一化后取补值 $\hat{s}(x) = 100 - \hat{s}(x)$ ，使高分始终对应用户偏好方向。

2. **布尔型参数**（如是否支持 4K 录制、是否具备主动降噪）按固定权值映射：支持赋值 100，不支持赋值 0。
3. **离散枚举型参数**（如传感器画幅类型：全画幅 > APS-C > M43 > 1 英寸）依据性能等级设置分档权重，确保不同规格之间具备可比的数值基础。

以相机品类为例，其评分体系融合了通用维度（性价比权重 0.20、品牌信誉 0.15、用户评价 0.15）与专有维度（低光画质 0.40、便携性 0.30、视频能力 0.30），通用与专有维度分别归一化后联合计算最终得分 $S(p)$ 。这一得分既作为候选排序依据，也以归一化形式直接驱动前端雷达图与多维柱状图的数值轴。

3. 双层场景化推荐引擎

在商品评分体系建立之后，推荐系统需要从大量商品中召回最符合用户需求的候选结果。如果仅依赖数值排序，往往难以体现数码产品选购中的经验知识；而完全依赖固定规则，又可能在预算约束较严格时出现无推荐结果的情况。

为提高系统稳定性，本研究设计了双层场景化推荐引擎。第一层为专家知识预设机制（Golden Sets）。系统将常见消费场景与优质机型建立映射关系，例如“电竞游戏”“户外 Vlog 拍摄”“高保真音乐欣赏”等。当用户需求中的字段 `usage` 与这些场景匹配时，系统优先在对应候选机型集合中进行筛选，从而利用领域经验快速缩小搜索范围。

然而，由于市场价格变化或预算限制，预设机型可能被全部排除。为避免推荐结果为空，系统设计了降级检索机制（Fallback Search）。当预设集合在预算过滤后剩余商品数量不足时，系统自动触发全库检索。此时推荐算法不再依赖场景规则，而是根据用户最关注的排序字段 `sort_field` 对数据库商品进行排序，并结合预算条件筛选新的候选商品。

最终，系统通过集合并集与去重策略整合两类结果，确保每次推荐均能输出 3 至 5 款不同商品，从而在保证推荐质量的同时提高系统鲁棒性。

4. 跨品类生态推荐算法

随着智能设备种类的增加，用户在购买单一设备时往往同时关注设备之间的协同体验。例如，在选择笔记本电脑时，用户可能还需要显示器、蓝牙音箱或无线鼠标等配套设备。因此，系统在核心推荐流程之外设计了跨品类生态推荐模块（Eco-system Suggestion）。

该模块首先构建若干设备生态主题集合，例如“移动办公生态”和“家庭影音娱乐生态”。每个主题包含一个核心设备类别以及若干关联设备类别。例如在移动办公场景中，笔记本电脑作为核心设备，而显示器和蓝牙扬声器等设备作为扩展组件。

在算法实现上，系统采用关键词权重评分机制对用户需求进行生态分类。例如与办公场景高度相关的关键词（如“编程”“出差”）被赋予较高权重，而“文档”等通用词汇则赋予较低权重。系统根据关键词累计得分判断用户所属的设备生态类别。

在确定生态类别后，系统结合用户已有设备列表 `owned_items` 进行过滤，以避免推荐重复设备。例如当用户已拥有显示器时，系统将优先推荐其他辅助设备。最终，系统通过语言模型生成简要推荐说明，并附加在主推荐结果之后，从而形成完整的设备组合建议。

算法 4 双层场景化推荐检索策略算法（对应文件：base_agent.py）

输入：user_message (String), category_agent (Object)

输出：candidate_products (List)

```
1: intent_data  $\leftarrow$  ParseIntentGeneric(user_message, category_agent.metadata)
2: target_scenario  $\leftarrow$  MatchScenarioRules(intent_data.usage, intent_data.summary)
3: candidate_products  $\leftarrow$  EMPTYLIST()
   {第一层：专家知识场景召回}
4: if target_scenario  $\neq$  NULL then
5:   preset_pool  $\leftarrow$  GetPresetProductsFromDatabase(target_scenario)
6:   filtered_pool  $\leftarrow$  FilterByBudgetAndBrand(preset_pool, intent_data.max_price,
       intent_data.brand)
7:   candidate_products  $\leftarrow$  SortByDimension(filtered_pool, intent_data.sort_field)
8:   candidate_products  $\leftarrow$  Limit(candidate_products, TOP=3)
9: end if
   {第二层：全库降级检索兜底}
10: if Length(candidate_products) < 3 then
11:   exclude_ids  $\leftarrow$  GetProductIDs(candidate_products)
12:   fallback_pool  $\leftarrow$  QueryDatabase(
13:     PRICE  $\leq$  intent_data.max_price,
14:     BRAND matches intent_data.brand,
15:     EXCLUDEIDS in exclude_ids,
16:     SORTBY = intent_data.sort_field,
17:     LIMIT = 3
18:   )
19:   candidate_products  $\leftarrow$  Merge(candidate_products, fallback_pool)
20:   candidate_products  $\leftarrow$  Limit(candidate_products, 3)
21: end if
22: return candidate_products = 0
```

```

1 ECOSYSTEMS = {
2     "移动办公": {
3         "name_en": "mobile_office",
4         "description": "适合移动办公、远程协作的数码产品组合",
5         "core_categories": ["laptop"], # 核心设备
6         "related_categories": ["monitor", "bluetooth_speaker", "
smartwatch"], # 关联外设
7         "scenarios": ["办公", "会议", "出差", "远程工作"],
8         "keywords": {
9             "high": ["办公", "工作", "商务", "出差", "会议", "远
程"], # 高权重关键词
10            "medium": ["文档", "表格", "演示", "ppt"] # 中权重关
键词
11        }
12    },
13    "专业游戏": {
14        "name_en": "professional_gaming",
15        "core_categories": ["laptop", "gaming_console", "gpu"],
16        "related_categories": ["monitor", "headphone"],
17        "scenarios": ["3A游戏", "电竞", "主机游戏", "PC游戏"],
18        # ...
19    }
20    ... ..
21 }

```

代码 3 跨品类生态环境定义与关联规则（对应文件：category_detector.py）

系统目前预置了六类生态应用场景（定义于 ecosystem_config.py），其核心设备与关联外设的映射关系如表3所示：

生态推荐的触发采用关键词权重评分机制：用户输入中高权重关键词（如“游戏”“电竞”）每次命中计 10 分，中权重关键词（如“打游戏”“帧率”）每次命中计 5 分，累计得分最高的前两个生态场景被选中。系统随后提取用户意图中的 owned_items 字段，将已有设备从关联品类候选中排除，避免重复推荐；对于核心设备与关联设备之间的优先级，系统赋予核心设备的关联推荐权重（+2）高于关联设备的反向推荐权重（+1），确保跨品类建议的相关性。最终，大语言模型根据生态场景描述、当前品类信息与用户诉求，生成一段自然、有针对性的搭配建议附加于主推荐结果之后，引导用户自然延伸至配套设备的选购。

表 3 生态推荐场景配置总览

场景名称	核心品类	关联配套品类	典型关键词
移动办公	笔记本电脑	显示器、蓝牙音箱、智能手表	办公、出差、会议
移动娱乐	手机、平板电脑	蓝牙音箱、耳机、智能手表	追剧、听歌、阅读
专业游戏	笔记本/显卡/游戏主机	显示器、耳机	游戏、电竞、3A
摄影创作	相机、笔记本、显卡	显示器、平板电脑	摄影、剪辑、后期
音频发烧	耳机、蓝牙音箱	手机、平板电脑	音质、HiFi、无损
健康运动	智能手表	耳机、手机	跑步、健身、运动

（三）系统架构实现

1. 全栈基础架构与运行环境实现

在第四章完成系统总体架构设计之后，本章进一步从工程实现角度说明系统的具体落地过程。系统实现阶段的核心目标是在保证功能完整性的前提下，使系统具备良好的可维护性与扩展能力。因此，在技术选型和架构实现过程中，本研究重点考虑开发效率、系统稳定性以及后期扩展需求。

在整体技术架构方面，系统采用 Python 作为主要开发语言，并构建了一套轻量化的全栈应用环境。其中，前端界面与交互逻辑基于 Streamlit 框架实现。Streamlit 可以通过 Python 脚本直接构建 Web 界面，从而显著降低前端开发复杂度。同时，该框架提供了基于状态驱动的界面刷新机制，使系统能够在用户输入发生变化时自动更新页面内容。

在系统交互过程中，用户的对话记录以及部分运行状态信息通过 `st.session_state` 进行统一管理。

如下所示的代码片段展示了应用入口状态的初始化逻辑，该机制能够在多轮交互中保持数据一致性，从而保证推荐流程能够持续依据用户输入进行动态更新：

在智能推理部分，系统通过标准 API 接口接入大语言模型服务。该接口遵循 Ope-

算法 5 系统状态初始化机制（对应文件：app.py）

```
0: {确保关键组件在多次交互中持久存在}
0: if central_router_agent  $\notin$  session then
0:   session[central_router_agent]  $\leftarrow$  Instantiate(MultiCategoryAgent)
0: end if
0: if conversation_history  $\notin$  session then
0:   session[conversation_history]  $\leftarrow$  EMPTYLIST()
0: end if
0: if current_recommendation_results  $\notin$  session then
0:   session[current_recommendation_results]  $\leftarrow$  NULL
0: end if
0: if active_visualization_charts  $\notin$  session then
0:   session[active_visualization_charts]  $\leftarrow$  NULL
0: end if
=0
```

nAI API 的调用规范，并通过配置文件统一管理模型参数，例如 config.LLM_MODEL 和自定义 baseUrl 等。具体实现见如下代码：

通过这种方式，模型调用逻辑与系统业务代码保持相对独立，当需要升级或替换底层模型时，只需修改配置文件即可完成迁移。

在数据可视化方面，系统采用 Plotly 作为主要图表绘制工具。Plotly 支持浏览器端交互式图表展示，能够实现动态缩放、悬停提示等功能，使用户可以更加直观地观察产品参数差异和评分结果。

系统的数据存储采用 SQLite 关系型数据库，并通过 SQLAlchemy ORM 框架进行管理。ORM 技术能够将数据库表结构映射为 Python 类对象，从而避免直接编写 SQL 语句所带来的安全隐患，同时也提高了代码的可读性和可维护性。

在项目结构组织方面，系统入口文件为 app.py，主要负责前端界面逻辑与会话控制；商品类别识别模块位于 category_detector.py；而各类商品推荐逻辑则集中在 electronics_agents.py 中实现。通过这样的模块划分，系统各组件之间保持较低耦合度，从而提升整体架构的清晰性。

算法 6 大语言模型客户端初始化架构（对应文件：base_agent.py）

```
0: {初始化与大语言模型服务的连接}
0: self.llm_interface ← InstantiateLLMClient(
0:     API_Key = RetrieveFromConfig("API_KEY"),
0:     Base_Endpoint = "https://api.model-provider.com/v1",
0:     Timeout_Seconds = 30,
0:     Max_Retries = 3
0: )
0: {验证连接状态}
0: if ConnectionFails(self.llm_interface) then
0:     LogWarning("Model service unavailable. Fallback protocols
0:         enabled.")
0: end if
0: =0
```

2. 多 Agent 推荐模块实现

在实际电商场景中，不同类别商品的评价标准往往存在较大差异。例如，相机产品通常更加关注画质表现和视频能力，而手机产品则更强调性能与续航能力。如果所有推荐逻辑集中在同一模块中，不仅代码结构复杂，也不利于系统后期扩展。

为了解决这一问题，系统在实现阶段采用了面向对象编程思想，并构建了多 Agent 推荐模块。系统首先在 base_agent.py 文件中定义了抽象基类 BaseProductAgent，用于描述所有商品推荐代理的共同行为。该基类通过 @abstractmethod 声明若干必须由子类实现的方法，例如获取数据库模型类型以及返回评分维度信息等，其核心结构如下：

在此基础上，系统在具备数据处理能力的基类 BaseDbAgent 中实现了统一的推荐流程方法 handle_chat_generic。该方法采用模板方法模式，将完整推荐流程划分为以下核心代码呈现的多个步骤：

通过这种方式，不同商品类别在实现时只需关注自身特有的评分逻辑，而无需重复编写完整流程代码。

在具体实现中，系统定义了多个继承自基类的子类。例如 CameraAgent 主要负责相机类产品推荐，而 PhoneAgent 则用于手机产品推荐。这些子类通过重写方法定义各自的评价指标与专属设置，例如 CameraAgent 定义的代码如下：

算法 7 推荐代理基类抽象接口定义（对应文件：base_agent.py）

```
1: Abstract Class BaseProductAgent:
2:   {所有商品共享的通用评分维度}
3:   Constant UNIVERSAL_DIMENSIONS  $\leftarrow$  [ Value_Score, Brand_Score,
      User_Rating ]
4:
5:   {需由具体品类代理实现的抽象方法}
6:   Abstract Function GetCategoryConfig()  $\rightarrow$  CategoryConfig
7:   Abstract Function GetSpecificDimensions()  $\rightarrow$  List<ScoringDimension>
8:   Abstract Function GetDatabaseSchema()  $\rightarrow$  DatabaseModel
9:   Abstract Function FormatProductInformation(product)  $\rightarrow$  String
10:
11:   {具体的推荐流转管道模板}
12:   Function ExecuteRecommendationPipeline(user_input, intent)  $\rightarrow$  Result:
13:     candidates  $\leftarrow$  FetchCandidates(intent)
14:     filtered  $\leftarrow$  FilterCandidates(candidates)
15:     sorted  $\leftarrow$  SortCandidates(filtered, intent.sort_field)
16:   Return sorted = 0
```

算法 8 核心商品推荐流转控制序列（对应文件：base_agent.py）

输入：self, user_message, history, db_model

输出：(justifications, visual_charts, ranked_results, analytical_text)

{步骤 1: 语义意图提取}

1: intent_data $\leftarrow$ Execute(parse_intent_generic, user_message)

{步骤 2: 提取硬性约束与软性偏好}

2: budget_limit $\leftarrow$ intent_data.max_price

3: target_scenario $\leftarrow$ MatchScenario(intent_data.usage)

{步骤 3: 候选商品集生成与排序}

4: candidate_products $\leftarrow$ GetPreconfiguredSet(target_scenario)

5: ranked_results $\leftarrow$ ApplyFilteringAndSorting(candidate_products, intent_data)

{步骤 4: 兜底机制 (若结果少于 3 款)}

6: **if** Count(ranked_results) < 3 **then**

7: ranked_results $\leftarrow$ Execute(fallback_full_database_search, intent_data)

8: **end if**

{步骤 5: 可解释性生成与可视化构建}

9: justifications $\leftarrow$ GenerateExplanationsWithLLM(ranked_results, user_message, intent_data)

10: visual_charts, analytical_text $\leftarrow$ GenerateVisualizations(ranked_results, intent_data)

11: **return** (justifications, visual_charts, ranked_results, analytical_text) = 0

算法 9 相机代理类实现 (Part 1: 配置与基础结构)

```
0: Class CameraAgent Inherits BaseDbAgent
0: {相机推荐代理}
0: Properties:
0: SCENARIO_PRESETS = {
0:     "vlog": ["ZV-E10", "G7 X", "Pocket", "Action", "Z30"],
0:     "travel": ["X100", "GR III", "a6400", "Z fc", "X-T30",
0:         "X-S10"],
0:     "street": ["GR III", "X100", "Leica", "Pen-F", "X-E4"],
0:     "portrait": ["A7", "R6", "R5", "Z6", "Z5", "5D"],
0:     "landscape": ["A7 R", "Z7", "D850", "GFX"],
0:     "beginner": ["R50", "Z30", "M50", "200D", "D3500", "a6000"]
0: }
0:
0: SCENARIO_KEYWORDS = {
0:     "vlog": ["vlog", " 视频", " 拍片", " 直播", "up 主"],
0:     "travel": ["travel", " 旅行", " 旅游"],
0:     "street": ["street", " 街拍", " 人文", " 扫街"],
0:     "portrait": ["portrait", " 人像", " 写真"],
0:     "landscape": ["landscape", " 风光", " 风景", " 大片"],
0:     "beginner": ["beginner", " 新手", " 入门", " 小白", " 学生"]
0: }
0:
0: Function get_category_config()
0:     =0
```

算法 10 相机代理类实现 (Part 2: 核心函数实现)

```
0: Function get_category_config() → CategoryConfig
0:   Return CategoryConfig(
0:     name = " 相机",
0:     name_en = "camera",
0:     table_name = "cameras",
0:     scoring_dimensions = self.get_specific_dimensions(),
0:     scenario_presets = self.SCENARIO_PRESETS,
0:     scenario_keywords = self.SCENARIO_KEYWORDS,
0:     default_sort_field = "LowLight_Score",
0:     display_fields = ["Brand", "Model", "Price",
0:       "LowLight_Score", "Video_Score", "Portability_Score"]
0:   )
0:
0: Function get_specific_dimensions() → List
0:   Return [
0:     ScoringDimension(" 便携性", "Portability_Score", 0.3, " 如重
0:       量体积"),
0:     ScoringDimension(" 低光画质", "LowLight_Score", 0.4, " 暗光表
0:       现"),
0:     ScoringDimension(" 视频能力", "Video_Score", 0.3, " 视频拍摄能
0:       力")
0:   ]
0:
0: Function get_model_class()
0:   Return Camera
0:
0: Function get_product_info_text(p)
0:   Return Concatenate(
0:     " 型号:", p.Brand, " ", p.Model,
0:     ", 价格:", p.Price,
0:     ", 低光:", p.LowLight_Score,
0:     ", 视频:", p.Video_Score,
0:     ", 便携:", p.Portability_Score
0:   )
0:
```

此处定义的维度直接映射到系统对相机进行综合打分的策略中，而手机类则会相似地关注“性能表现”和“续航能力”等指标并为其分配比重。

为了实现不同商品类别之间的统一调度，系统在 `multi_agent.py` 中实现了中央调度模块 `MultiCategoryAgent`。该模块作为全系统的单一访问入口，负责将非结构化的用户查询路由至最合适的领域专家代理。其核心调度逻辑（如算法 11 所示）涵盖了品类识别、代理调度、动态代理构建以及跨品类生态建议生成等关键环节。

当系统接收到用户消息后，首先由 `CategoryDetector` 识别目标品类（`Category Key`）。若该品类已预置了专家代理（如 `CameraAgent`），则直接进行任务分发；若属于长尾品类，系统将激活动态代理构建机制（`create_dynamic_agent`），利用大语言模型推演该品类的评分维度并生成通用选购建议，从而保证了系统对未知品类的泛化处理能力。

此外，该模块还承载了系统的“生态系统增强”功能。在获取主品类的推荐结果后，系统会分析用户意图中的已购设备（`owned_items`）并调用生态配置模块，识别用户当前的潜在应用场景（如“移动办公”或“游戏发烧”）。通过大语言模型生成自然的生态搭配建议，系统能够主动推荐具有强互补性的次选品类，将原本单一的商品筛选过程升华为场景化的整体解决方案构建。

通过这种方式，系统能够灵活支持多种商品类型，并且在需要扩展新类别时只需新增对应 Agent 并注册，无需修改总体处理链路。

3. 品类识别与动态路由实现

在 `MultiCategoryAgent` 的调度框架下，`CategoryDetector` 承担着品类识别的核心职责，其工作流程分为两个层次。

第一层：关键词快速匹配。系统预先为 10 个品类维护了关键词词典，覆盖中英文通俗名称与常见缩写。每次接收到用户输入后，系统对词典中所有品类的关键词进行遍历匹配，若用户输入与某品类关键词存在包含关系则命中该品类。例如输入“索尼微单”会命中“相机”品类，输入“PS5”会命中“游戏主机”品类。关键词匹配的时间复杂度为 $O(K \cdot L)$ （ K 为所有品类关键词总数， L 为用户输入长度），在实测中可在毫秒级完成识别。

第二层：LLM 语义推断。当关键词匹配未能命中任何预设品类时，系统将用户输入传递给大语言模型，要求其从已知品类列表中选择最匹配的品类，或标记为未知品类（`is_new: True`）。对于未知品类，系统进一步调用 `build_scoring_system()` 方法，引导 LLM 逐步构建该品类的评分维度、典型使用场景及推荐关键词。这一动

算法 11 多品类中央路由分发算法（对应文件：multi_agent.py）

输入： 用户消息 user_message (String), 对话历史 conversation_history (List)

输出： 建议原因 reasons, 可视化图表 charts, 商品列表 products, 数据分析 analyses, 生态建议 eco_suggestion

```
1: (category_key, category_name) ← detector.detect_category(user_message)
2: target_agent ← GetRegisteredAgent(category_key)
3: if target_agent = NULL then
4:   target_agent ← _create_dynamic_agent(category_key, category_name,
     user_message)
5:   if target_agent = NULL then
6:     return Error("Unable to build handling agent")
7:   end if
8: end if
9: (reasons, charts, products, analyses) ← target_agent.handle_chat(user_message, his-
   tory)
   {生态系统增强逻辑}
10: eco_info ← get_ecosystem_recommendations(user_message, category_key)
11: if eco_info 匹配到关联品类 then
12:   owned_items ← ExtractFromAgentIntent(target_agent)
13:   eco_suggestion ← _generate_eco_suggestion_llm(..., owned_items)
14: end if
15: return (reasons, charts, products, analyses, eco_suggestion) = 0
```

态扩展机制使系统在保持代码不变的前提下，能够对" 扫地机器人"" 洗碗机"" 电子书阅读器" 等长尾品类给出基础的选购建议，显著提升了系统的泛化能力。

在已知品类命中后,MultiCategoryAgent 查询 _agents 字典中对应品类的 Agent 实例。由于所有预置 Agent (CameraAgent、PhoneAgent、LaptopAgent 等 10 个) 在系统启动时统一完成初始化并注册，调度过程仅为一次字典查询，延迟可忽略不计。若字典中不存在对应键（即动态新品类），系统才通过 _create_dynamic_agent() 即时构建通用 Agent 实例并缓存至 _dynamic_configs 字典，供后续同品类请求复用，避免重复调用 LLM 推演评分体系。

4. 商品数据层实现与数据清洗

推荐系统的有效性在很大程度上依赖于底层数据质量。因此，在系统实现过程中，本研究构建了结构规范的商品数据库，并通过数据清洗保证数据的准确性和一致性。

系统数据库通过 SQLAlchemy ORM 框架进行管理，所有数据模型统一定义在 models.py 文件中。各类商品数据表通过继承 Base 类建立实体对象，例如 Camera、Laptop 等类分别对应数据库中的商品数据表结构。

设计数据模型时，每个类别都包含基础属性如品牌 (Brand)、型号 (Model) 和价格 (Price)。同时针对不同商品类别，系统还设计了专门的参数字段和评分维度，例如通过 SQLAlchemy 定义的 Camera 模型：

如上所示，手机数据表也会有自己的电池容量 (Battery_mAh) 等指标，这些结构化数据与评分参数能够为推荐算法在生成策略时提供更加细粒度的参考信息。

当专家知识库中的场景预案无法满足用户的预算约束或特定偏好时，系统会激活兜底搜索机制。该机制通过 BaseProductAgent 类中的 fallback_search 方法实现，利用 SQLAlchemy 提供的 ORM 接口对完整事实数据库执行带约束的动态检索。

如算法 13 所示，检索过程分为三个阶段。首先是应用硬性约束过滤，系统根据意图解析得到的 max_price 字段对所有商品进行价格阈值筛选；其次是进行去重处理，排除掉第一阶段已识别出的推荐项，以保证推荐结果的多样性；最后是动态排序阶段，系统检测目标数据表是否具备用户偏好的排序字段 (sort_field)，若存在则执行降序排列，否则退而求其次使用系统默认的维度 (default_sort_field)。

ORM 框架会自动将这些链式调用操作转换为底层经过优化的 SQL 查询语句。这种实现方式不仅保证了数据检索的安全性，全面规避了 SQL 注入风险，还使系统能够灵活响应大语言模型提取的动态意图，在多维数据向量空间中精准平衡预算与

算法 12 关系型数据库表的结构化表示（对应文件：models.py）

1: **Database Schema Definition: Table "Cameras"**
2:
3: PrimaryKey: id (Integer)
4:
5: **Standard_Attributes:**
6: Brand (String) { 品牌标识}
7: Model (String) { 产品型号名称}
8: Price (Float) { 当前市场价格}
9: Year (Integer) { 发布年份}
10: Image_File (String) { 产品图片路径}
11:
12: **Technical_Specifications:**
13: Total_Megapixels (Float) { 传感器分辨率}
14: Sensor_Type (String) { 画幅类型（全画幅、APS-C 等）}
15: Weight_g (Float) { 设备重量}
16: Supports_4K (Boolean) { 4K 视频录制支持情况}
17:
18: **Scoring_Metrics (离线计算得到):**
19: Portability_Score (Float) [0-100]
20: LowLight_Score (Float) [0-100]
21: Video_Score (Float) [0-100] =0

算法 13 事实数据约束与动态检索过滤（对应文件：base_agent.py）

输入： 数据库会话 database_session, 意图数据 intent_data, 排除 ID 列表 exclude_ids

输出： 商品列表（最多 3 条）

```
1: Model ← GetModelClass()
2: search_query ← database_session.query(Model)
   {阶段 1: 硬性预算约束过滤}
3: price_field ← GetPriceFieldName(Model)
4: search_query ← search_query.filter( Model.price_field ≤ intent_data.max_price )
   {阶段 2: 排除已命中项（去重）}
5: if exclude_ids 不为空 then
6:   search_query ← search_query.filter( ~Model.id.in(exclude_ids) )
7: end if
   {阶段 3: 意图驱动的动态排序}
8: sort_field ← intent_data.sort_field or Default_Sort_Field
9: if Model 拥有 sort_field then
10:   search_query ← search_query.order_by( desc(sort_field) )
11: end if
12: return search_query.limit(3).all()=0
```

性能。

在数据层构建阶段，系统对原始数据进行了必要的清洗处理。原始商品数据来源于网络爬虫抓取的 CSV 文件，其中普遍存在字段格式不一、量纲缺失等工程问题。为此，系统设计了自动化的批处理管道：首先去除参数值中的特殊字符与单位前缀（如“g”、“mAh”），将其转换为可计算的数值类型；其次针对“支持/不支持”等文本描述，统一映射为布尔型标识；最后处理缺失值并完成数据正则化。经过清洗后的规范数据集通过 SQLAlchemy 的 SessionLocal 管道导入数据库，确立了系统的“单一事实来源”，从根源上约束了大模型产生幻觉的空间。表4对系统当前支持的 10 类商品数据模型进行了汇总，展示了各品类关键技术参数字段与专属评分维度的结构化建模方式：

所有品类均共享三个通用评分维度（性价比权重 0.20、品牌信誉 0.15、用户评价 0.15），与专属维度合并后形成完整的评分向量。得益于 SQLAlchemy ORM 的抽象层，新增品类仅需在 models.py 中定义继承 Base 的新数据类，并在 electronics_agents.py 中实现对应 Agent 子类，即可无缝接入既有推荐流程，无需改动数据库连接与推荐链路的任何代码。

5. 用户界面与交互式可视化实现

为了提高推荐系统的可用性，本研究在界面设计中加入了可视化展示功能，使用户能够更加直观地理解产品之间的性能差异。如果仅以文本形式呈现大量参数信息，用户往往难以快速完成比较，因此可视化展示在系统中具有重要作用。

系统前端界面基于 Streamlit 的状态驱动机制实现。当推荐模块返回结果后，界面会根据 st.session_state 中存储的数据自动刷新显示内容，从而形成类似单页应用的交互体验。

在商品展示快显方面，系统设计了统一的商品卡片组件 render_product_card()。该组件通过 Python 的 f-string 模板引擎构建 HTML 片段，并结合外部 CSS 样式表实现玻璃拟态（Glassmorphism）视觉效果：

使用这种基于 HTML 模板的内嵌组件方案，极大方便在 Streamlit 等响应式数据大屏中动态组合不同类别的特征信息，使页面布局既保留了 Web 开发的灵活性，又兼顾了数据驱动的高效性。

系统针对不同商品类别设计了差异化的参数展示策略。在 render_product_card() 函数内部，通过判断 SQLAlchemy 对象的类名（class_name）自动切换规格标签内容：手机展示处理器型号、RAM+ROM 组合、电池容量和主摄像头；笔记本展示 CPU

表 4 10 类商品数据模型关键字段与专属评分维度汇总

品类	关键技术参数字段	专属评分维度
相机	总像素 (MP)、传感器类型、最高 ISO、重量 (g)、4K 支持	便携性、低光画质、视频能力
手机	处理器、RAM/ROM、电池容量 (mAh)、主摄像头、操作系统	性能、拍照、续航
笔记本	CPU、GPU、RAM、屏幕尺寸、重量 (kg)、续航 (h)、类别	性能、便携性、屏幕
耳机	类型 (头戴/入耳)、无线、主动降噪、续航 (h)、驱动 (mm)、阻抗 ()	音质、舒适度、降噪效果
平板	屏幕尺寸、处理器、RAM/ROM、电池 (mAh)、手写笔支持	性能、屏幕、生产力
智能手表	续航 (天)、防水等级、操作系统、健康功能	续航、健康功能、智能体验
蓝牙音箱	功率 (W)、蓝牙版本、续航 (h)、防水等级、单元配置	音质、续航、便携性
显示器	尺寸 (in)、分辨率、刷新率 (Hz)、面板类型、色域、响应时间 (ms)	画质、性能、人体工学
游戏主机	存储容量、光驱、最大分辨率、最高帧率 (fps)、GPU 算力 (TF)	游戏性能、媒体能力、性价比
显卡	VRAM(GB)、显存带宽 (GB/s)、TDP(W)、3DMark 跑分、架构	游戏性能、创作性能、功耗散热

算法 14 基于状态驱动的视图组件渲染机制（对应文件：app.py）

输入： product_entity, justification_text

输出： 无（前端渲染）

{提取核心属性并格式化}

1: brand, model $\leftarrow$ ExtractInfo(product_entity)

2: price_str $\leftarrow$ Format(product_entity.Price)

{动态生成规格与评分标签}

3: specs $\leftarrow$ ParseCategorySpecs(product_entity)

4: top_scores $\leftarrow$ SelectTopThreeScores(product_entity)

5: score_html $\leftarrow$ JoinTags(top_scores, class="score-tag")

6: spec_html $\leftarrow$ JoinTags(specs, class="spec-tag")

{注入大模型生成的推荐理由}

7: reason_html $\leftarrow$ ""

8: **if** justification_text $\neq$ NULL **then**

9: reason_html $\leftarrow$ FormatReason(justification_text, class="reason-box")

10: **end if**

{构建并注入 HTML 模板}

11: template $\leftarrow$ f-string("<div class='product-card'> ...</div>")

12: st.markdown(template, unsafe_allow_html=True) = 0

型号、独立显卡、内存规格和屏幕尺寸；耳机展示类型、是否支持主动降噪（ANC）和续航时长；相机展示总像素、最高 ISO、机身重量和 4K 支持情况；其余品类类似处理。这种按需渲染策略确保每张商品卡片都以最贴合决策需求的参数维度呈现信息，有效降低用户在信息密集界面中的认知负荷。

商品卡片的评分标签部分同样经过动态筛选：系统遍历产品对象所有以 `_Score` 结尾的属性，提取评分最高的前三项生成彩色得分标签置于卡片顶部，快速传递该产品最突出的性能亮点。

在视觉设计层面，系统通过外部 `style.css` 文件实现了玻璃拟态（Glassmorphism）界面风格。主题配色方案采用静谧蓝（#5D89B1）、金黄（#F1C40F）与橙红（#E67E22）三色组合，同时应用于雷达图与柱状图的轨迹颜色，使可视化图表与整体界面风格保持视觉统一。所有图表通过 `_apply_theme()` 函数统一注入白色底板、`plotly_white` 模板和 `Inter` 字体族，保证图表视觉质感的一致性。Plotly 内置的鼠标悬停提示、图例点击切换与坐标轴缩放等交互特性无需额外开发即可提供流畅的动态交互体验，有效降低了可视化模块的工程实现成本。

在数据可视化部分，系统通过 Plotly 绘图引擎提供了三类交互式图表，旨在全方位展示产品性能差异并增强结果的可解释性。

第一类为雷达图（Radar Chart），通过 `go.Scatterpolar` 函数绘制。系统将商品的各维度原始参数进行 Min-Max 归一化处理（映射至 0-100 区间），并以填充面积的形式展示单个产品在多个评分维度上的综合表现。这种呈现方式便于用户直观观察产品各项能力的平衡度，例如观察能力面积的大小以及在特定维度（如便携性或画质）上的偏向程度。图表支持交互式缩放，并在悬停时显示精确的维度分值。

第二类为单指标柱状对比图（Bar Chart），由 `draw_comparison` 函数实现。该图表专门用于针对用户最关心的某一特定参数（如价格、综合得分或特定的性能评分）进行横向直观对比。图表采用了系统预设的浅色粉紫主题配色方案（Theme Colors），并自动在柱状顶部标注数值，以便于用户快速捕捉不同产品在单一核心指标上的优劣性。

第三类则是多维能力对比图，通过指定 `barmode='group'` 参数实现分组柱状对比。相较于雷达图侧重于单产品的“画像”，分组柱状图更强调多款候选产品在同一组核心指标（如便携性、低光画质、视频能力等）上的性能层级对比。用户可以通过交互式的图例切换（Legend Toggle）动态隐藏或显示特定的候选产品，并通过悬停提示框（Hover Tooltip）获取更详尽的规格说明。

这些图表不仅在视觉上采用了玻璃拟态（Glassmorphism）与现代配色风格，更

在逻辑上与后端的双层推荐引擎紧密联动，实时反映动态生成的评分数据，帮助用户在复杂的数码产品配置中快速做出理性的购买决策。

算法 15 实现了多商品雷达图的绘制逻辑。通过 Plotly 的 Scatterpolar 轨迹组件，系统将每个商品的多维评分映射到极坐标多边形，并通过 `fill='toself'` 填充多边形内部，使不同商品的维度轮廓差异一目了然。首尾数值重复 (`values += values[:1]`) 确保多边形闭合，`opacity=0.2` 的半透明填充则允许多个商品的雷达图叠加显示而互不遮挡，有效呈现各商品在不同维度上的综合优劣势。

算法 16 实现了单维度得分的柱状图对比。函数接收一个字段名参数(如 `LowLight_Score`)，提取各商品对应字段的数值后，以统一的品牌主题色绘制柱状图，并在柱体顶端自动标注具体分值。纵轴范围固定为 `[0, 110]`，确保跨查询结果的视觉一致性，避免因动态缩放导致用户对绝对分值产生视觉误判。

算法 17 实现了多维度分组柱状图的绘制，采用 `barmode='group'` 将同一维度下各商品的得分紧邻排列，便于横向对比。相较于雷达图侧重整体轮廓，分组柱状图在各维度的精确数值比较上更具优势，两者在系统中组合使用，可同时满足用户对“宏观印象”与“局部细节”的可视化需求，形成互补的双视图决策支持结构。

(四) 本章小结

本章从需求分析、核心算法与系统实现三个层面对智能商品推荐系统进行了完整阐述。

第一节通过 20 份有效问卷梳理了用户在电商选购中的核心痛点——信息来源分散 (85% 用户需跨平台切换)、评价主观性强 (80% 用户认为网络评测存在偏向)、技术参数难以理解 (40% 用户希望图表辅助)，并据此提炼出自然语言理解、多品类推荐、结构化筛选、推荐理由生成、可视化对比和跨品类搭配六项功能需求，以及响应时延、识别准确率、可扩展性和可解释性四项非功能需求。在架构层面，提出了由表象展示层 (`app.py`)、路由意图层 (`multi_agent.py`、`category_detector.py`)、领域决策层 (`base_agent.py`、`electronics_agents.py`) 和事实数据层 (`models.py`) 组成的四层架构，并设计了涵盖 10 个品类、按中类分组的三层分类体系，为后续模块实现提供了清晰的组织框架。

第二节展开算法设计。在语义解析方面，采用“角色设定 + 结构化输出”的复合提示策略，提取 8 个槽位信息，结合多轮对话上下文历史 (`history[-2:]`) 与容错回退机制，实现了稳定的自然语言到结构化参数的转换；针对未知品类，引入思维链诱导策略动态推演评分体系。在评分模型方面，建立了加权求和公式 $S(p) = \sum w_i \hat{s}_i(p)$,

算法 15 雷达图绘制算法 (`draw_radar`, 对应文件: `visualizer.py`)

输入: `products` (推荐商品列表), `dimensions` (可选维度配置)

输出: 多维雷达对比图对象 `fig`

```
1: {步骤 1: 确定雷达轴标签}
2: if dimensions  $\neq$  NULL then
3:   labels  $\leftarrow$  [“便携”, “画质”, “视频”, “高感”, “操控”]
4: else
5:   labels  $\leftarrow$  [d[0] for d in dimensions]
6: end if
7: {步骤 2: 为每个商品生成极坐标轨迹}
8: for i, p in ENUMERATE(products) do
9:   values  $\leftarrow$  []
10:  for each (label, field, max_val) in dimensions do
11:    v  $\leftarrow$  GETATTR(p, field, 0)
12:    if max_val  $\neq$  0 then
13:      v  $\leftarrow$   $\min(v/max\_val \times 100, 100)$ 
14:    end if
15:    APPEND(values, v)
16:  end for
17:  values  $\leftarrow$  values + values[0:1] {首尾闭合多边形}
18:  color  $\leftarrow$  THEMECOLORS[i mod 3]
19:  ADDTRACE(fig, SCATTERPOLAR(r=values, theta=labels, fill=“toself”, opacity=0.2,  

      line_color=color))
20: end for
21: {步骤 3: 应用全局主题与图例}
22: APPLYTHEME(fig) {白底, Inter 字体, 蓝色刻度}
23: UPDATELAYOUT(fig, polar.radialaxis.range=[0,100], showlegend=True, height=320)
24: return fig = 0
```

算法 16 单维度柱状图对比 (`draw_comparison`, 对应文件: `visualizer.py`)

输入: `products` (推荐商品列表), `field_name` (对比字段名, 如 `LowLight_Score`)

输出: 单维度柱状对比图对象 `fig`

```
1: {步骤 1: 提取商品简称与分数}
2: names ← [Model[0:8] + “.” for each product]
3: full_names ← [Brand + Model for each product]
4: scores ← [GETATTR(p, field_name, 0) for p in products]
5: {步骤 2: 构造柱状图对象}
6: fig ← FIGURE(BAR(
7:   x = names, y = scores,
8:   marker_color = THEMECOLORS[0:LEN(products)],
9:   text = scores, textposition = “auto”,
10:  hovertext = full_names
11: ))
12: {步骤 3: 格式化标题与坐标范围}
13: safe_name ← REPLACE(field_name, “_Score”, “评分”)
14: APPLYTHEME(fig)
15: UPDATELAYOUT(fig, title=safe_name + “对比”, yaxis.range=[0, 110], height=300)
16: return fig = 0
```

算法 17 多维分组柱状图对比 (draw_multi_dimension_compare, 对应文件: visualizer.py)

输入: products (推荐商品列表), dimensions (可选维度配置)

输出: 多维分组柱状对比图对象 fig

```
1: {步骤 1: 确定比较维度列表}
2: if dimensions  $\neq$  NULL then
3:   dims_conf  $\leftarrow$  [(“便携性”, Portability_Score), (“低光画质”,
   LowLight_Score), (“视频能力”, Video_Score)]
4: else
5:   dims_conf  $\leftarrow$  dimensions
6: end if
7: labels  $\leftarrow$  [d[0] for d in dims_conf]
8: fields  $\leftarrow$  [d[1] for d in dims_conf]
9: {步骤 2: 为每个商品生成一组柱}
10: for i, p in ENUMERATE(products) do
11:   scores  $\leftarrow$  [GETATTR(p, f, 0) or 0 for f in fields]
12:   color  $\leftarrow$  THEMECOLORS[i mod 3]
13:   ADDTRACE(fig, BAR(name=p.Model, x=labels, y=scores, marker_color=color,
   text=scores))
14: end for
15: {步骤 3: 应用分组模式与主题}
16: APPLYTHEME(fig)
17: UPDATELAYOUT(fig, barmode=“group”, title=“核心能力多维对比”, yaxis.range=[0,
   110], height=320)
18: return fig = 0
```

针对连续、布尔、枚举三种数据类型分别设计了 Min-Max 归一化、二值映射和分档权重三种归一化策略，确保异质参数在统一评分空间内可比。在推荐引擎方面，双层机制（场景预设 + 全库兜底）保证了在严苛约束下的推荐鲁棒性；跨品类生态推荐模块基于六大场景配置与关键词权重评分，在单品推荐完成后自动触发配套建议。

第三节将设计落地为可运行系统。全栈环境基于 Python + Streamlit + SQLite + DashScope API 搭建，通过 `st.session_state` 实现多轮对话的状态持久化。多 Agent 模块以面向对象设计实现，10 个预置领域 Agent 在启动时统一注册，动态品类通过即时构建的通用 Agent 处理。数据层覆盖 10 类商品共数千条参数记录，经自动化清洗管道处理后形成“单一事实来源”。可视化模块提供雷达图、单指标柱状图与多维分组柱状图三类交互式图表，配合类别自适应的商品卡片组件和玻璃拟态界面风格，将推荐结果以图文结合的形式直观呈现给用户。各模块设计环环相扣，共同验证了本文提出方案在理论设计与工程实现两个层面的完整可行性。

四、系统评估

本章的工作重心落在验证上——用自动化脚本和多维度功能测试来检验系统是否真的好用、逻辑是否真的严密。评估严格围绕项目源代码中内嵌的验证体系展开，重点考察三个方面：意图解析的准确度、推荐逻辑能否形成闭环，以及可视化输出的联动效果是否符合预期。

（一）评估方案设计

系统的可靠性需要分层次地去验证。本研究为各核心模块设计了对应的自动化测试脚本，将定性判断转化为可量化的评估结果。

验证工作部署于配备 16GB RAM 的本地工作站（CPU 为 Intel i7 系列），软件栈基于 Python 3.10 构建。推理层接入阿里云 DashScope 平台的 Qwen 系列模型（走 OpenAI 兼容协议），事实数据库层采用 SQLite 3。

验证标准从两个层面展开。组件级单元验证借助 `verify_multi_category.py` 脚本，分别对 `BaseProductAgent` 基类、`CameraAgent` 专家代理、`CategoryDetector` 品类识别器和 `MultiCategoryAgent` 路由中心进行连通性与逻辑完整性测试；全链路流转验证则由 `verify_agent_flow.py` 等脚本模拟真实对话，追踪从用户输入到系统输出——涵盖推荐理由、Plotly 图表与数据分析文案——的完整链路是否畅通。

在性能基准方面，回归测试设定了两条硬性要求：对于“相机”“笔记本”等预设

商品品类关键词，识别器的准确率必须达到 100%；在网络条件正常的情况下，从意图解析到输出三张联动图表（Radar、Bar、Multi-dim）的端到端耗时应控制在 3 至 5 秒以内。

（二）核心功能逻辑验证

为检验系统在极端条件下的健壮性，本节结合项目内的压力测试脚本，重点测试了两类边界场景。

第一类是基于降级检索的边界条件测试，测试用例参考 `test_recommendation.py`：输入极低预算（如 2000 元），但要求推荐高性能相机或笔记本。系统的处理路径如下：`electronics_agents.py` 中的 `_filter_and_sort` 方法首先对专家预设集执行硬性过滤，若过滤后符合预算的产品不足三款，则自动触发 `_fallback_search` 兜底逻辑，依据 `intent` 中的 `max_price` 字段重新发起 SQLAlchemy 数据库查询。这一测试直接验证了双层推荐机制的容灾能力——系统在严苛约束下不会因“无匹配数据”而中断对话，也不会凭空捏造型号，而是退而求其次，从全库中找出性价比最高的备选方案。

第二类是动态品类泛化能力验证，测试对象为 `CategoryDetector` 面对非内置品类（如“扫地机”）时的表现。结果显示，系统能准确识别出 `is_new: True` 标识并激活 `DynamicAgent` 的维度推演逻辑，借助 LLM 的常识推理自发生成了适用于“扫地机”场景的评估维度（如清扫能力、避障系数等）。这说明系统的品类处理能力并非只在预设范围内有效，面对长尾需求同样具备相当的弹性。

（三）系统效能评估分析

与传统关键词查询相比，纯粹的参数过滤在面对模糊表达时往往力不从心。`_parse_intent_generation` 建立的意图映射层能在这类语境下准确抽取 `usage`（用途）和 `summary`（诉求摘要），将用户需求更精准地对齐至数据库中的性能向量轴，这是结构化提取相比纯文本检索的核心优势所在。

可视化层面，`visualizer.py` 生成的动态图表在可解释性与信息密度两个维度上均表现突出。`_get_individual_reasons` 方法将晦涩的数值对比转化为易懂的自然语言描述，让推荐结果“有据可查”；Plotly 引擎将多维参数归一化至 0-100 区间，生成的雷达图使用户无需深究参数含义，便可快速感知不同产品在便携性、性能、续航等维度上的相对位置。两者配合，使系统的输出结果从“给出答案”进一步迈向“解释答案”。

（四）从输入到图表的终局案例演示

本节选取三个典型测试用例，完整呈现系统从意图捕捉到图表反馈的处理过程。

1. 案例 1：大学生的轻薄办公场景（笔记本电脑）

场景预设

用户 A 是一名刚入学的大学生，对电脑硬件参数了解不多。其需求较为明确：日常在图书馆写论文、看视频，需要频繁携带外出，对续航和便携性较为敏感，预算在 6000 元以内。他在输入框中写道：

“轻薄本，续航一定要好，看视频写论文，6000 元以下。”

交互与推理过程

系统接收请求后，首先通过 CategoryDetector 的语义匹配逻辑，从“写论文”、“图书馆”等关键词中精准识别出 laptop（笔记本电脑）品类。随后，MultiCategoryAgent 将任务路由至 LaptopAgent。

在意图解析阶段，LLM 识别出“续航一定要好”这一强约束偏好，并结合“看视频写论文”的使用描述，将场景锁定为 light_office（轻薄办公）。系统随即触发预设的权重分配方案：将性能评分（Performance_Score，权重 0.4）、便携评分（Portability_Score，权重 0.3）与屏幕评分（Display_Score，权重 0.3）作为多维评价的核心。由于用户明确提及“6000 元以下”的预算限制，系统在意图对象中将 max_price 硬性锁定为 6000，并将 Value_Score（性价比评分）设为默认排序字段，以平衡性能表现与资金投入。

结果呈现

系统推荐了三款机型：联想小新 Air 14 (€4,299)、宏碁非凡 Go Pro (€4,999) 与华硕天选 Air (€5,999)。三款产品的便携性评分均为 92.0，在用户最关注的核心维度上不相上下；差异主要体现在其余维度上——小新 Air 14 性价比评分以 94.0 居首，屏幕评分为 85.0；非凡 Go Pro 性价比同为 94.0，屏幕评分提升至 88.0，但售价高出 700 元；天选 Air 凭借 AMD R9-7940HS 处理器与 RTX 4050 独显将性能评分拉至 86.0，但性价比评分相应降至 92.0，售价也达到预算上限的 5,999 元。

系统为每款产品生成了简短的场景化评语，直接说明产品与用户使用场景的契合点，避免堆砌参数。深度对比分析报告进一步提供了三个层次的横向比较：综合能力雷达图呈现四维能力分布，核心评分分布专项对比便携性数据，多维对比则明

确指出各款产品的相对优势——“天选 Air 在性能表现最佳；小新 Air 14 在性价比表现最佳；非凡 Go Pro 在屏幕表现最佳”——将取舍判断的依据清晰交还给用户。

此外，系统在页面末尾附上了生态链延伸建议：“如果轻薄本主要用来处理和剪辑素材，可以搭配一台色彩准确的显示器。”该建议并非用户此次咨询的核心诉求，但体现了系统对用户潜在使用场景扩展的主动预判，在完成即时推荐任务的同时为后续可能的追加咨询提供了自然的引导入口。



图 1 笔记本电脑推荐结果与对比分析

2. 案例 2：极致性能的游戏发烧场景（跨品类生态组合）

场景预设

用户 B 是一名资深游戏玩家，预算充裕，追求顶级游戏体验。市面上顶级配件太多，组合方式繁复，他不想花几个周末去挨个研究兼容性和最新发布的型号，也不希望花费大量精力逐一核查各配件的兼容性与搭配关系，倾向于获得完整的配置建议。他在输入框中写道：

“不差钱，可以玩顶级 3A 游戏的显卡。”

第一轮交互：显卡推荐

系统通过 GPUAgent 处理该请求。解析过程中，系统识别到用户输入的“不差钱”关键词，触发了针对极端性能词汇的补丁逻辑（Patch Logic）：将 max_price 阈值上调至 999,999 元，并自动将最优排序字段（sort_field）映射为该品类最核心的创作性能维度（Creative_Score），以确保推荐结果聚焦于顶级规格。

最终推荐结果包含三款产品：NVIDIA GeForce RTX 4060 Ti 16GB（€3,499）、Intel Arc B770（€2,799）与 NVIDIA GeForce RTX 4060 Ti（€3,199）。RTX 4060 Ti 16GB 与 Arc B770 在游戏与创作性能评分上完全持平（均为 80.0/82.0），但在功耗散热评分（Thermal_Score）上，RTX 4060 Ti 16GB 以 92.0 领先。系统在可视化雷达图中完整呈现了 B770 更高的 3DMark 原始跑分，但基于驱动稳定性与生态成熟度的综合博弈，最终将 16GB 版本的 RTX 4060 Ti 列为首选推荐。

完成单品推荐后，系统调用 EcosystemConfig 模块，识别出“游戏”场景下的跨品类关联需求。通过 _generate_eco_suggestion_llm 函数，系统生成了极具引导性的专业建议：“都上顶级显卡了，画面必须拉满才过瘾！配一台 4K 高刷显示器，

能完全发挥这块显卡的潜力。”这一建议成功激发了用户对显示输出端性能对等性的关注。

第二轮交互：显示器推荐

用户 B 在看到生态链建议后，认识到显示器与显卡的配套关系值得进一步明确，随即追加输入：

“配台高分辨率、高刷新率的电竞显示器。”

系统将品类切换至显示器，由 MonitorAgent 接手检索，推荐结果包含三款产品：LG 27GR95QE (€9,999, 240Hz / 2560€1440)、LG 27GR950 (€4,999, 144Hz / 3840€2160) 与技嘉 M27Q (€2,299, 170Hz / 2560€1440)。三款产品人体工学评分均为 88.0，差异集中在画质、性能与性价比三个维度：LG 27GR95QE 画质与性能评分均达 98.0，综合素质最高，但性价比评分仅为 68.0，溢价明显；LG 27GR950 画质评分 96.0、性能评分 95.0，以 4K 分辨率提供了更高的画面精细度；技嘉 M27Q 画质评分 94.0、性能评分 94.0，在三款中性价比最为突出。

系统综合分析指出：27GR95QE 在画质与人体工学表现最佳，27GR950 在性能维度最优，并建议用户根据具体需求权衡选择。同时，系统将技嘉 M27Q 作为性价比导向的重点推荐，指出其 27 英寸 2K 分辨率搭配 170Hz 刷新率的组合能够在合理预算内大幅提升游戏体验。

两轮交互完成后，用户 B 从最初模糊的“顶级游戏显卡”需求，经由生态链建议的自然引导，逐步落地为一套显卡与显示器协同考量的完整配置方案。这一过程体现了系统跨品类推荐链路的设计逻辑：用户无需预先规划所有配件，系统在完成当前品类推荐后主动识别关联需求，以生态链建议作为跨品类跳转的触发机制，将单品咨询有序延伸为场景化的整体方案。

~~image.png~~ ~~image=1.png~~ ~~image=2.png~~

图 2 显卡与显示器推荐结果及生态链建议

3. 案例 3：追求性价比的摄影入门场景（相机）

场景预设

用户 C 最近迷上了在社交平台上看别人的 Vlog，开始有了自己记录生活的想法，觉得手机拍出来的画面总差点意思，想买一台相机试试。她在网上查了一些攻略，但“机身防抖”“相位对焦”“4K 30fps”这些词让她越看越迷糊，攻略里动辄出现的参数对比表也让她不知道该从哪里入手判断。她的需求其实并不复杂：预算 5000 元左

右，拍日常 Vlog，希望拍出来的画面稳、对焦不拉风箱、操作别太复杂。她在系统里写道：

“5000 元以内拍 Vlog 的相机。”

交互与推理过程

系统识别出“Vlog”关键词后，将其映射为 CameraAgent 中的 vlog 场景预设。推理引擎自动从 SCENARIO_PRESETS 中调取热门候选机型（如 ZV-E10, Pocket, Action 等），并针对该特定场景分配权重：视频能力（Video_Score）占比 0.3，便携性（Portability_Score）占比 0.3，而将暗光表现（LowLight_Score）权重设为最高（0.4），旨在筛选出能够胜任全天候记录的器材。由于预算限制在 5000 元以内，系统过滤了昂贵的专业全画幅型号，优先展示轻量化、高集成度的解决方案。

结果呈现

系统推荐了三款产品：大疆 Osmo Pocket 3 (€3,499)、大疆 Osmo Action 4 (€2,199) 与佳能 PowerShot G7 X Mark III (€4,200)。三款产品呈现出明显差异化的能力分布：Osmo Pocket 3 视频能力评分最高（95.0），便携性评分达 98.0，机身仅重 179g，在场景核心维度上综合表现最为突出，系统将其列为首选；Osmo Action 4 便携性评分最优（99.0），机身最轻（145g），ISO 上限更高（12800），更适合户外动态拍摄，且售价仅 2,199 元，性价比突出；PowerShot G7 X Mark III 低光画质评分以 93.86 大幅领先另外两款，但便携性评分仅为 29.2，机身重量达 304g，与另外两款形成鲜明对比，其能力分布与 Vlog 入门场景的核心诉求存在一定错位。

上述差异在雷达图中得到了完整呈现。三款产品在便携性轴线上的分布尤为直观——G7 X Mark III 的便携性短板在图形上一目了然，无需用户逐一比对重量数值即可直观感知。系统进一步在多维对比中明确指出各款产品的相对优势：“Osmo Action 4 在便携性表现最佳；PowerShot G7 X Mark III 在低光画质表现最佳；Osmo Pocket 3 在视频能力表现最佳”，将不同优先级下的取舍逻辑清晰呈现，辅助用户根据自身实际拍摄场景完成最终判断。

系统还在结果页末尾附上了生态链建议：“拍 Vlog 要效率高，配个能随时预览和剪辑的平板电脑会更顺手——相机直连传输素材，大屏剪辑比手机舒服很多。”该建议将推荐视野从单一设备延伸至创作工作流，对有意持续投入内容创作的用户具有实际参考价值，同时也为后续可能的追加咨询提供了引导入口。



图 3 相机推荐结果与多维对比

（五）本章小结

通过项目内嵌的验证脚本，本章对系统进行了系统性评估。结果表明，Agent 协同架构在应对复杂消费意图、处理极端预算约束以及生成联动可视化反馈三个方面均表现良好。整体技术链路的闭环性符合预期，系统确实能够将繁琐的产品参数转化为直观的视觉决策依据。

五、本文贡献总结

电商平台的商品信息历来以参数为主要载体，用户在下单前往往需要自行消化大量技术指标，与真实购物决策逻辑之间存在明显落差。本文以此为出发点，提出并实现了一套基于大型语言模型与多智能体协同的对话式数码产品推荐系统，主要工作可从以下四个方面加以归纳。

（一）提出面向消费场景的对话式推荐范式

传统电商平台将产品信息拆解为参数条目逐一陈列，对普通消费者形成隐性门槛。本文将大型语言模型的自然语言理解能力与多智能体任务分发机制整合，以多轮对话为主要交互形式构建推荐路径。用户无须预先了解产品规格，只需用日常语言描述使用需求，系统便在对话过程中逐步完成意图识别、方案筛选与结果解释。

（二）构建兼顾灵活性与可靠性的双层推荐架构

大型语言模型擅长理解模糊表达，但若直接用于生成具体商品推荐，“幻觉”问题便难以规避。对此，本文采取双层处理策略：前一层由大模型负责意图解析与结构化查询生成，后一层将查询结果交由本地 SQLAlchemy 数据库核验执行，确保每条推荐记录均有据可查。当标准查询路径无法覆盖用户意图时，回退搜索引擎作为补充介入。这种分工在保留语义理解灵活性的同时，将幻觉风险限定在推荐结果之外。

（三）实现跨品类协同推荐

消费者的实际采购需求往往并不孤立——学生选购笔记本时多半同时需要耳机与移动电源，摄影爱好者升级机身时镜头与存储方案同样在考虑之列。本文为此设

计了 MultiCategoryAgent 路由模块与 EcosystemConfig 生态配置，使系统具备跨品类识别与联动推荐的能力，将原本分散的选购环节整合为一次连贯的决策过程。

(四) 以可视化手段增强推荐的可解释性

推荐结果能否获得用户信任，很大程度上取决于用户是否理解推荐的依据。本文在呈现层面引入基于 Plotly 的多类型动态可视化组件：雷达图用于多维性能的直观对比，柱状图用于单项指标的横向比较，多维散点图则允许用户从价格、性能、口碑等维度同时审视候选产品。推荐理由以平实语言输出，与图形内容相互补充，让用户不只是拿到一个结论，也能看懂结论从何而来。

六、局限性

(一) 商品数据的实时性不足

当前系统的商品数据存储于本地 SQLite 数据库，更新依赖离线爬取，难以与电商平台保持同步。促销期间价格往往在数小时内完成调整，库存状态的变化更是持续发生。若用户依据系统呈现的旧价格做出决策，结账时才发现价格已有偏差，推荐的实用性便会大打折扣。接入京东联盟等电商开放平台的官方 API 是较直接的改进路径，可在实现数据动态更新的同时，减少对自行爬取的依赖。

(二) 复杂语义场景下的解析局限

CategoryDetector 处理常规模糊表达时运行稳定，但面对隐含前提较强的表达方式时容易出现偏差，例如以反讽传达偏好，或借助多重否定界定需求。这类说法在日常对话中并不少见，通用大模型对消费语境下的情感语义缺乏专项适配，处理时容易遗漏关键信息。引入经过细粒度情感分析微调的模型是一个可行方向，但需结合具体数据集评估，单纯提升模型规模未必奏效。

(三) 个体偏好的持续建模缺失

系统当前的推荐权重来自人工预设的场景规则，反映的是群体层面的共性特征，对个体差异无从捕捉。用户对品牌的使用习惯、对外观的审美偏向，乃至对某类参数的特殊敏感，在现有架构中均付之阙如。每次会话从零开始，多次交互之间也不

存在任何积累。在隐私保护前提下引入轻量级个人偏好模型，是系统走向实用化绕不开的问题。

七、后续改进方向

其一，引入 RAG 以补充软性评价信息。硬件参数之外，握持手感、系统流畅度、售后口碑等使用体验对消费决策的影响同样不可忽视。将评测视频文字稿与用户评价纳入外挂知识库，可使推荐在参数比较之外兼顾真实反馈。

其二，建立用户操作的反馈闭环。重新生成方案、修改预算、反复对比某两款产品——这些操作本身都隐含着对当前推荐的评价信号。若能将其系统性地收集并用于模型校准，系统便有条件从固定规则逐步过渡到数据驱动的个性化策略。

其三，将推荐引擎与 Streamlit 前端解耦，以标准 API 形式独立部署，进而分别接入微信小程序与多设备客户端。小程序无需安装、触达成本低；多端流转则允许用户在手机上提问、电脑上深入比较、平板上查看可视化结果，各端衔接而不中断。

本文所构建的系统目前仍处于原型验证阶段，主要意义在于打通一条将大语言模型语义理解与数据库事实约束相结合的技术路径，并在可解释性呈现上做了初步探索。已识别的局限性，自然构成后续工作的起点。

参考文献

- [1] Adomavicius G, Tuzhilin A. Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions. *IEEE Transactions on Knowledge and Data Engineering*, 2005, 17(6): 734~749.
- [2] Schafer J B, Frankowski D, Herlocker J, *et al.* Collaborative Filtering Recommender Systems. In: *The Adaptive Web*. Berlin: Springer, 2007. 291~324.
- [3] Koren Y, Bell R, Volinsky C. Matrix Factorization Techniques for Recommender Systems. *Computer*, 2009, 42(8): 30~37.
- [4] Pazzani M J, Billsus D. Content-Based Recommendation Systems. In: Brusilovsky P, Kobsa A, Nejdl W, eds. *The Adaptive Web*. Berlin: Springer, 2007. 325~341.
- [5] Burke R. Knowledge-based Recommender Systems. In: *Encyclopedia of Library and Information Systems*. 2000.
- [6] Burke R. Hybrid Recommender Systems: Survey and Experiments. *User Modeling and User-Adapted Interaction*, 2002, 12(4): 331~370.
- [7] Bao K, Zhang J, Zhang Y, *et al.* TALLRec: An Effective and Efficient Tuning Framework to Align LLMs with Recommendation. In: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2023.
- [8] Zhang Y, *et al.* Collaborative Large Language Model for Recommender Systems. *arXiv preprint arXiv:2311.01343*, 2023.
- [9] Ji Z, *et al.* Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 2023, 55(12).
- [10] Russell S, Norvig P. *Artificial Intelligence: A Modern Approach*. 4th ed. London: Pearson, 2020.
- [11] Wooldridge M. *An Introduction to MultiAgent Systems*. 2nd ed. Hoboken: Wiley, 2009.
- [12] Stone P, Veloso M. Multiagent Systems: A Survey from a Machine Learning Perspective. *Autonomous Robots*, 2000, 8(3): 345~383.
- [13] Hong S, *et al.* MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. *arXiv preprint arXiv:2308.00352*, 2023.

- [14] Park J S, O'Brien J, Cai C, *et al.* Generative Agents: Interactive Simulacra of Human Behavior. In: UIST '23: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. 2023.
- [15] Guidotti R, Monreale A, Ruggieri S, *et al.* A Survey of Methods for Explaining Black Box Models. *ACM Computing Surveys*, 2018, 51(5): 93:1~93:42.
- [16] Wang H, Zhang F, Wang J, *et al.* RippleNet: Propagating User Preferences on the Knowledge Graph for Recommender Systems. In: Proceedings of the 27th ACM International Conference on Information and Knowledge Management. 2018. 417~426.
- [17] Zhang Y, Chen X. Explainable Recommendation: A Survey and New Perspectives. *Foundations and Trends in Information Retrieval*, 2020, 14(1): 1~101.
- [18] Strobel H, Gehrmann S, Pfister H, *et al.* LSTMVis: A Tool for Visual Analysis of Hidden State Dynamics in Recurrent Neural Networks. *IEEE Transactions on Visualization and Computer Graphics*, 2018, 24(1): 667~676.
- [19] Wei J, *et al.* Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: Advances in Neural Information Processing Systems (NeurIPS). 2022.
- [20] Amar R, Eagan J, Stasko J. Low-level Components of Analytic Activity in Information Visualization. In: IEEE Symposium on Information Visualization. 2005. 15~21.
- [21] Zhang S, *et al.* A Novel Multivariate Data Visualization Tool that Improves Radar Chart. *BMC Medical Research Methodology*, 2023, 23(84).
- [22] Amershi S, Cakmak M, Knox W B, *et al.* Power to the People: The Role of Humans in Interactive Machine Learning. *AI Magazine*, 2014, 35(4): 105~120.
- [23] Cai C, *et al.* Generative User Interfaces: Designing Dynamic Interfaces with Large Language Models. 2024.
- [24] Baylor A L, Richman T. Gender and Expert–Novice Differences in the Perception of Pedagogical Agent Persona. *Computers in Human Behavior*, 2002, 18(2): 213~234.
- [25] Brown T B, Mann B, Ryder N, *et al.* Language Models are Few-Shot Learners. In: Advances in Neural Information Processing Systems, 2020, 33: 1877 1901.
- [26] Liu P, Yuan W, Fu J, *et al.* Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Computing Surveys*, 2021, 55(9): 1 35.

- [27] White J, Fu Q, Hays S, *et al.* A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. In: Proceedings of the 2023 Conference on Pattern Languages of Programs. 2023.

致 谢

感谢使用本模板。

Thanks for using this template.